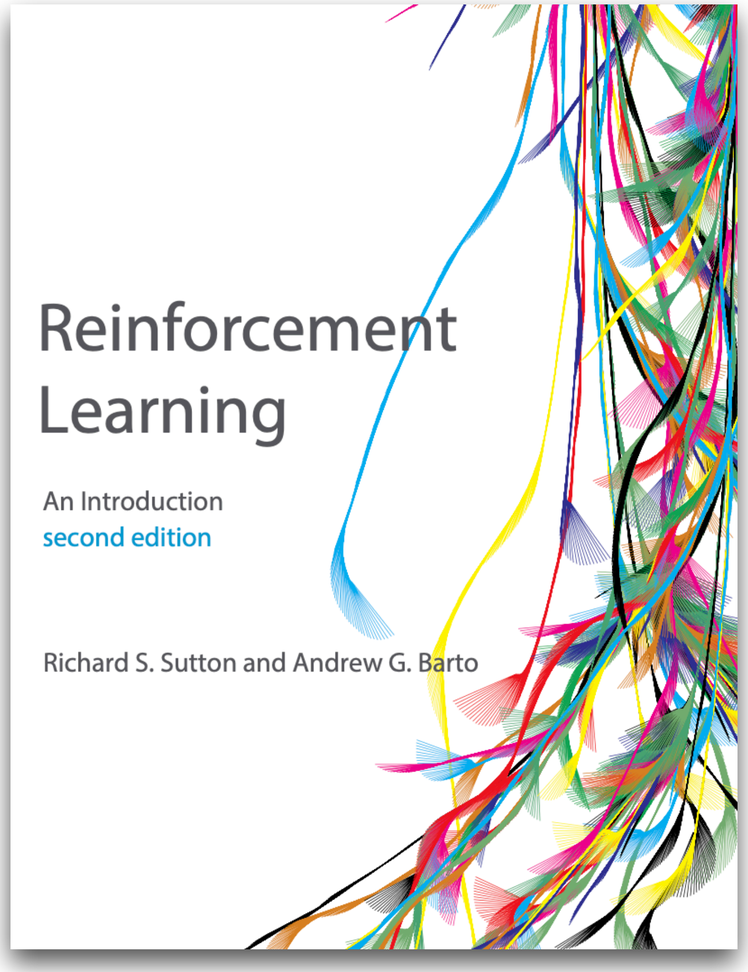
L9: Temporal Difference Learning

EECE 576N | From MDPs to Deep RL | Fall 2026

Cyrus Neary | cyrus.neary@ubc.ca



Chapter 6

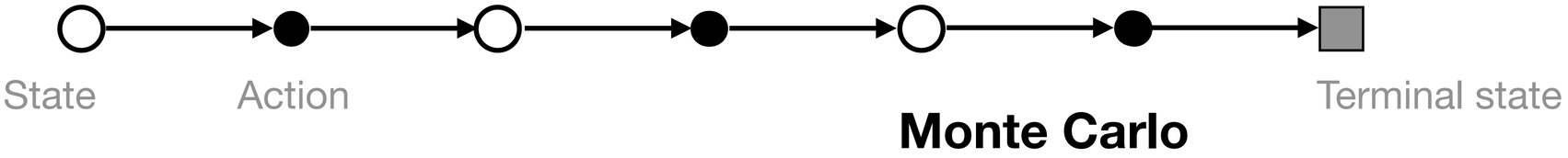
Temporal-Difference Learning

If one had to identify one idea as central and novel to reinforcement learning, it would undoubtedly be *temporal-difference* (TD) learning. TD learning is a combination of Monte Carlo ideas and dynamic programming (DP) ideas. Like Monte Carlo methods, TD methods can learn directly from raw experience without a model of the environment's dynamics. Like DP, TD methods update estimates based in part on other learned estimates, without waiting for a final outcome (they bootstrap). The relationship between TD, DP, and Monte Carlo methods is a recurring theme in the theory of reinforcement learning; this chapter is the beginning of our exploration of it. Before we are done, we will see that these ideas and methods blend into each other and can be combined in many ways. In particular, in Chapter 7 we introduce n-step algorithms, which provide a bridge

Last class: Monte Carlo methods for policy evaluation

Estimate $V^{\pi}(s) = \mathbb{E}\left[G_t \mid S_t = s, \pi\right]$ by sampling episodes, computing returns G_t along the **entire episode**, and averaging those returns.

$$G_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \ldots + \gamma^{T^{(i)}-1} R_{T^{(i)}-1}$$

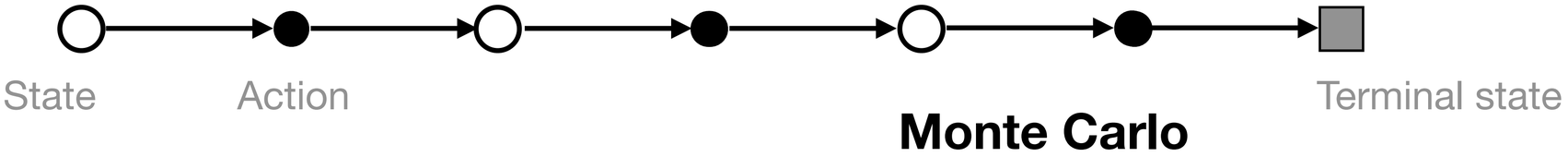


Monte Carlo

Last class: Monte Carlo methods for policy evaluation

Estimate $V^{\pi}(s) = \mathbb{E}\left[G_t \mid S_t = s, \pi\right]$ by sampling episodes, computing returns G_t along the **entire episode**, and averaging those returns.

$$G_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \ldots + \gamma^{T^{(i)}-1} R_{T^{(i)}-1}$$



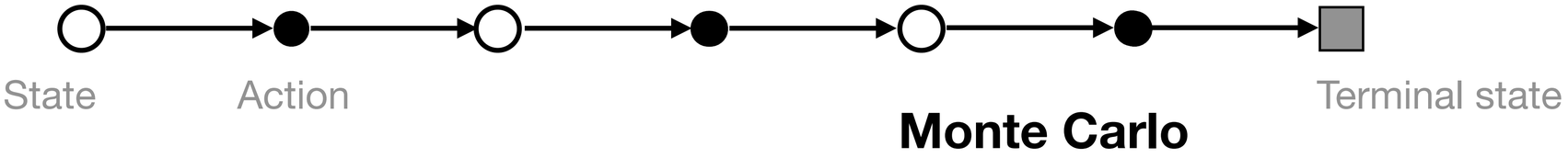
Monte Carlo

Note, from definition of value function, $V^{\pi}(s) = \mathbb{E}\left[G_t \mid S_t = s, \pi\right] = \mathbb{E}\left[R_t + \gamma V^{\pi}(s_{t+1}) \,|\, S_t = s, \pi\right]$.

Last class: Monte Carlo methods for policy evaluation

Estimate $V^\pi(s) = \mathbb{E}\left[G_t \mid S_t = s, \pi\right]$ by sampling episodes, computing returns G_t along the **entire episode**, and averaging those returns.

$$G_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \ldots + \gamma^{T^{(i)}-1} R_{T^{(i)}-1}$$



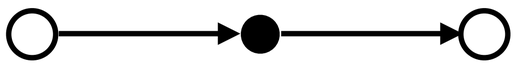
Monte Carlo

Note, from definition of value function, $V^\pi(s) = \mathbb{E}\left[G_t \mid S_t = s, \pi\right] = \mathbb{E}\left[R_t + \gamma V^\pi(s_{t+1}) \mid S_t = s, \pi\right]$.

Today: temporal difference methods

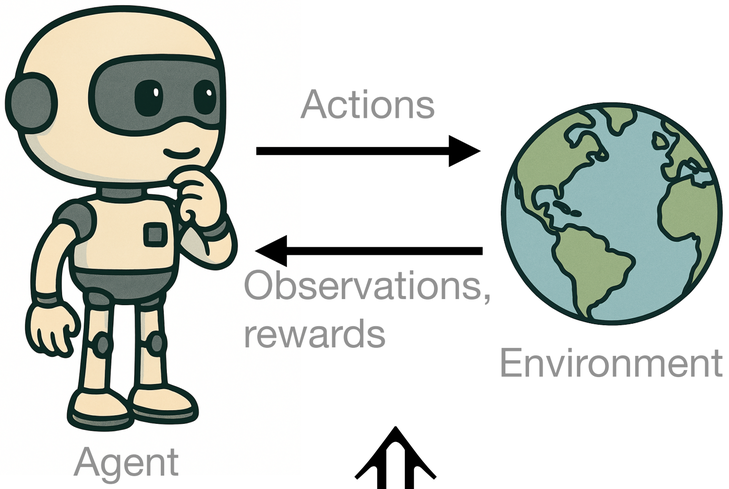
Estimate $V^\pi(s) = \mathbb{E}\left[R_t + \gamma V^\pi(s_{t+1}) \mid S_t = s, \pi\right]$ from **incomplete episodes** by *bootstrapping*.

$$R_t + \gamma \hat{V}^\pi(S_{t+1})$$



TD(0)

Monte-Carlo-based generalized policy iteration



$$\tau^1 = s_0^1, a_0^1, r_0^1, s_1^1, a_1^1, r_1^1, \ldots, s_{T_1}^1$$

$$\tau^2 = s_0^2, a_0^2, r_0^2, s_1^2, a_1^2, r_1^2, \ldots, s_{T_2}^2$$

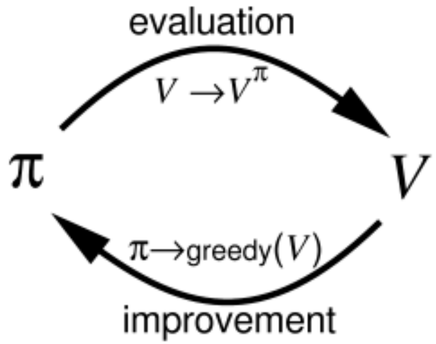


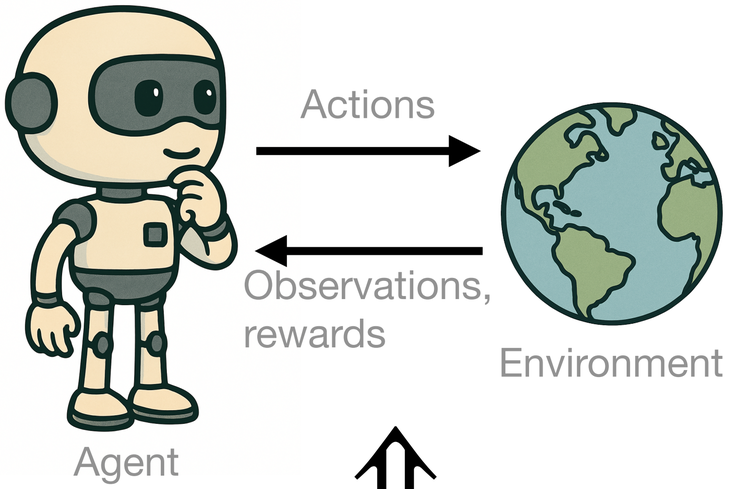
Figure courtesy of Sutton & Barto (2018)

Use estimate of $V^\pi(s)$ to improve $\pi \to \pi'$

Monte Carlo estimation of $\hat{V}^\pi(s)$

~~Monte-Carlo~~-based generalized policy iteration

~~Temporal difference~~



$$\tau^1 = s_0^1, a_0^1, r_0^1, s_1^1, a_1^1, r_1^1, \ldots, s_{T_1}^1$$

$$\tau^2 = s_0^2, a_0^2, r_0^2, s_1^2, a_1^2, r_1^2, \ldots, s_{T_2}^2$$

One-step transition + reward

$$s_t, a_t, r_t, s_{t+1}$$

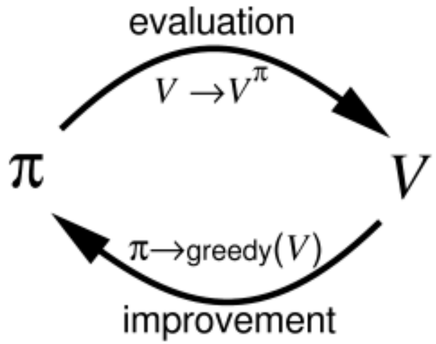


Figure courtesy of Sutton & Barto (2018)

Use estimate of $V^\pi(s)$ to improve $\pi \to \pi'$

~~Monte Carlo~~ **TD(0)** estimation of $\hat{V}^\pi(s)$

Aside: Running averages

Suppose we want to keep a running average of samples $x^{(i)}$ of a random variable X.

$$\hat{\mu}_n = \frac{1}{n}\sum_{i=1}^{n} x^{(i)} = \frac{1}{n}\left(x^{(n)} + \sum_{i=1}^{n-1} x^{(i)}\right)$$

Aside: Running averages

Suppose we want to keep a running average of samples $x^{(i)}$ of a random variable X.

$$\hat{\mu}_n = \frac{1}{n}\sum_{i=1}^{n} x^{(i)} = \frac{1}{n}\left(x^{(n)} + \sum_{i=1}^{n-1} x^{(i)}\right)$$

But, $\hat{\mu}_{n-1} = \frac{1}{n-1}\sum_{i=1}^{n-1} x^{(i)} \implies \sum_{i=1}^{n-1} x^{(i)} = (n-1)\hat{\mu}_{n-1}$

Aside: Running averages

Suppose we want to keep a running average of samples $x^{(i)}$ of a random variable X.

$$\hat{\mu}_n = \frac{1}{n}\sum_{i=1}^{n} x^{(i)} = \frac{1}{n}\left(x^{(n)} + \sum_{i=1}^{n-1} x^{(i)}\right)$$

But, $\hat{\mu}_{n-1} = \frac{1}{n-1}\sum_{i=1}^{n-1} x^{(i)} \implies \sum_{i=1}^{n-1} x^{(i)} = (n-1)\hat{\mu}_{n-1}$

So, $\hat{\mu}_n = \frac{1}{n}\left((n-1)\hat{\mu}_{n-1} + x^{(n)}\right) = \hat{\mu}_{n-1} + \frac{1}{n}\left(x^{(n)} - \hat{\mu}_{n-1}\right)$

Aside: Running averages

Suppose we want to keep a running average of samples $x^{(i)}$ of a random variable X.

$$\hat{\mu}_n = \frac{1}{n}\sum_{i=1}^{n} x^{(i)} = \frac{1}{n}\left(x^{(n)} + \sum_{i=1}^{n-1} x^{(i)}\right)$$

But, $\hat{\mu}_{n-1} = \frac{1}{n-1}\sum_{i=1}^{n-1} x^{(i)} \implies \sum_{i=1}^{n-1} x^{(i)} = (n-1)\hat{\mu}_{n-1}$

So, $\hat{\mu}_n = \frac{1}{n}\left((n-1)\hat{\mu}_{n-1} + x^{(n)}\right) = \hat{\mu}_{n-1} + \frac{1}{n}\left(x^{(n)} - \hat{\mu}_{n-1}\right)$

Running average formula: $\hat{\mu}_n = \hat{\mu}_{n-1} + \frac{1}{n}\left(x^{(n)} - \hat{\mu}_{n-1}\right)$

Aside: Running averages and tracking non-stationary problems

Running average formula: $\hat{\mu}_n = \hat{\mu}_{n-1} + \frac{1}{n}\left(x^{(n)} - \hat{\mu}_{n-1}\right)$

In RL, we often replace $\frac{1}{n}$ with a constant *step-size* parameter $\alpha \in (0,1]$.

$$\hat{\mu}_n = \hat{\mu}_{n-1} + \alpha\left(x^{(n)} - \hat{\mu}_{n-1}\right)$$

Aside: Running averages and tracking non-stationary problems

Running average formula: $\hat{\mu}_n = \hat{\mu}_{n-1} + \frac{1}{n}\left(x^{(n)} - \hat{\mu}_{n-1}\right)$

In RL, we often replace $\frac{1}{n}$ with a constant *step-size* parameter $\alpha \in (0,1]$.

$$\hat{\mu}_n = \hat{\mu}_{n-1} + \alpha\left(x^{(n)} - \hat{\mu}_{n-1}\right)$$

Current estimate

Previous estimate

step size

Update target

Aside: Running averages and tracking non-stationary problems

Running average formula: $\hat{\mu}_n = \hat{\mu}_{n-1} + \frac{1}{n}\left(x^{(n)} - \hat{\mu}_{n-1}\right)$

In RL, we often replace $\frac{1}{n}$ with a constant *step-size* parameter $\alpha \in (0,1]$.

$$\hat{\mu}_n = \hat{\mu}_{n-1} + \alpha\left(x^{(n)} - \hat{\mu}_{n-1}\right)$$

Current estimate

Previous estimate

step size

Update target

Interpretation: estimate $\hat{\mu}_n$ at update n is a weighted average of sampled values and the initial some initial guess $\hat{\mu}_0$.

$$\begin{aligned}
\hat{\mu}_{n+1} &= \hat{\mu}_n + \alpha\left(x^{(n)} - \hat{\mu}_n\right) \\
&= \alpha x^{(n)} + (1-\alpha)\hat{\mu}_n \\
&= \alpha x^{(n)} + (1-\alpha)\left(\alpha x^{(n-1)} + (1-\alpha)\hat{\mu}_{n-1}\right) \\
&= \alpha x^{(n)} + (1-\alpha)\alpha x^{(n-1)} + (1-\alpha)^2\hat{\mu}_{n-1} \\
&= (1-\alpha)^n\hat{\mu}_0 + \sum_{i=1}^{n}\alpha(1-\alpha)^i x^{(i)}
\end{aligned}$$

Do the math

Aside: Running averages and tracking non-stationary problems

Running average formula: $\hat{\mu}_n = \hat{\mu}_{n-1} + \frac{1}{n}\left(x^{(n)} - \hat{\mu}_{n-1}\right)$

In RL, we often replace $\frac{1}{n}$ with a constant *step-size* parameter $\alpha \in (0,1]$.

$$\hat{\mu}_n = \hat{\mu}_{n-1} + \alpha\left(x^{(n)} - \hat{\mu}_{n-1}\right)$$

Current estimate

Previous estimate

step size

Update target

Interpretation: estimate $\hat{\mu}_n$ at update n is a weighted average of sampled values and the initial some initial guess $\hat{\mu}_0$.

$$\begin{aligned}
\hat{\mu}_{n+1} &= \hat{\mu}_n + \alpha\left(x^{(n)} - \hat{\mu}_n\right) \\
&= \alpha x^{(n)} + (1-\alpha)\hat{\mu}_n \\
&= \alpha x^{(n)} + (1-\alpha)\left(\alpha x^{(n-1)} + (1-\alpha)\hat{\mu}_{n-1}\right) \\
&= \alpha x^{(n)} + (1-\alpha)\alpha x^{(n-1)} + (1-\alpha)^2\hat{\mu}_{n-1} \\
&= (1-\alpha)^n\hat{\mu}_0 + \sum_{i=1}^{n}\alpha(1-\alpha)^i x^{(i)}
\end{aligned}$$

The step size α is an exponential decay constant.

Similarly to how the discount factor γ decays the impact of future rewards on the value now, α decays the impact of past observed samples on the running average now.

The TD(0) (aka one-step TD) method for estimating $\hat{V}^{\pi}(s)$

Execute policy π in the environment. After every step (s_t, a_t, r_t, s_{t+1}), form a target and make an update of $\hat{V}^{\pi}(s)$.

Running average of samples of: $R_t + \gamma \hat{V}^{\pi}(S_t)$



The TD(0) (aka one-step TD) method for estimating $\hat{V}^\pi(s)$

Execute policy π in the environment. After every step (s_t, a_t, r_t, s_{t+1}), form a target and make an update of $\hat{V}^\pi(s)$.

Running average of samples of: $R_t + \gamma \hat{V}^\pi(S_t)$

$$\implies \hat{V}^\pi(s_t) \leftarrow \hat{V}^\pi(s_t) + \alpha \left(r_t + \gamma \hat{V}^\pi(s_{t+1}) - \hat{V}^\pi(s_t) \right)$$

The TD(0) (aka one-step TD) method for estimating $\hat{V}^\pi(s)$

Execute policy π in the environment. After every step (s_t, a_t, r_t, s_{t+1}), form a target and make an update of $\hat{V}^\pi(s)$.

Running average of samples of: $R_t + \gamma \hat{V}^\pi(S_t) \implies \hat{V}^\pi(s_t) = \hat{V}^\pi(s_t) + \alpha \left(\underbrace{r_t + \gamma \hat{V}^\pi(s_{t+1}) - \hat{V}^\pi(s_t)}_{\text{TD error}} \right)$

The TD(0) (aka one-step TD) method for estimating $\hat{V}^\pi(s)$

Execute policy π in the environment. After every step (s_t, a_t, r_t, s_{t+1}), form a target and make an update of $\hat{V}^\pi(s)$.

Running average of samples of: $R_t + \gamma \hat{V}^\pi(S_t) \implies \hat{V}^\pi(s_t) = \hat{V}^\pi(s_t) + \alpha \Big(r_t + \gamma \hat{V}^\pi(s_{t+1}) - \hat{V}^\pi(s_t) \Big)$

TD error

Tabular TD(0) for estimating v_π

Input: the policy π to be evaluated
Algorithm parameter: step size $\alpha \in (0, 1]$
Initialize $V(s)$, for all $s \in \mathcal{S}^+$, arbitrarily except that $V(terminal) = 0$

Loop for each episode:
  Initialize S
  Loop for each step of episode:
    $A \leftarrow$ action given by π for S
    Take action A, observe R, S'
    $V(S) \leftarrow V(S) + \alpha[R + \gamma V(S') - V(S)]$
    $S \leftarrow S'$
  until S is terminal

Figure courtesy of Sutton & Barto (2018)

How does the TD(0) update relate to Bellman's equations?

Bellman equations $V^\pi(s) = \sum_{a \in A} \pi(a|s) \left[R(s,a) + \gamma \sum_{s' \in S} T(s'|s,a) V^\pi(s') \right]$

$= \mathbb{E}\left[R(S_t, A_t) + \gamma V^\pi(S_{t+1}) \mid S_t = s, \pi \right]$

TD(0): Using sampled transitions (s_t, a_t, r_t, s_{t+1}) to compute a running estimate of the expectation in the Bellman equation.

$\hat{V}^\pi(s) \leftarrow \hat{V}^\pi(s) + \alpha \left(r + \gamma \hat{V}^\pi(s') - \hat{V}^\pi(s) \right)$

Bootstrapping: TD(0) is a bootstrapping method because it bases its update target on an existing estimate.



Defining TD error

Note that the TD(0) target $r_t + \gamma \hat{V}^\pi(s_{t+1}) - \hat{V}^\pi(s_t)$ is just the difference between value estimate now $\hat{V}^\pi(s_t)$ and the better estimate $r_t + \gamma \hat{V}^\pi(s_{t+1})$, which includes the value estimate one timestep into the future.

We often refer to the quantity

$$\delta_t = r_t + \gamma \hat{V}^\pi(s_{t+1}) - \hat{V}^\pi(s_t)$$

as the TD error, which comes up often in RL algorithms.



What advantages do TD methods have over DP and Monte Carlo methods?

DP:
- Efficiency
  - ↳ DP must iterate over entire state-action space to get one update.
  - ↳ TD methods "local" updates around our samples.
- knowledge vs. sampling
  - ↳ DP needs a model of $T(s'|s,a)$ and $R(s,a)$
  - ↳ TD only requires samples.

MC:
- MC methods require rollouts of entire trajectories to compute updates.
  - ↳ TD doesn't require us to run to the end of the episode.
    - ↳ Episodes could be long. Maybe we want to update part way through.
      - ↳ "focus" on certain parts of a trajectory.



TD(0) example



Experiment setup from Sutton and Barto (2018) pg. 125.

- No actions.
- Start in state C.
- Equal probability of transitioning either left or right at each step.
- Absorbing (terminal) states on the left and the right.
- Reward +1 for reaching the right.
- Reward 0 for everything else.
- No discounting (i.e. $\gamma = 1$).

True value function: probability of reaching the right-most terminal state.

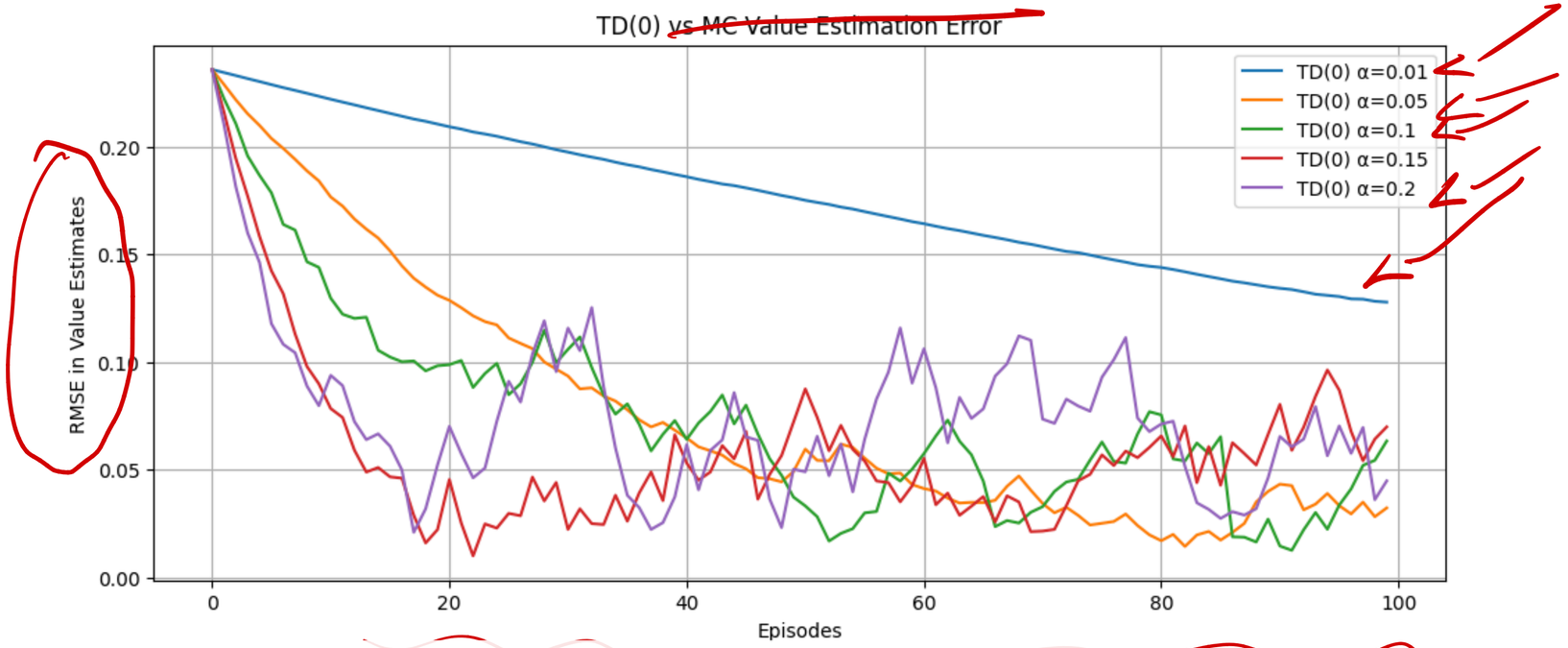
$$V(s_A) = \frac{1}{6},\ V(s_B) = \frac{1}{3}\ \ V(s_C) = \frac{1}{2},\ V(s_D) = \frac{2}{3}\ V(s_E) = \frac{5}{6}$$

TD(0) example



Experiment setup from Sutton and Barto (2018) pg. 125.

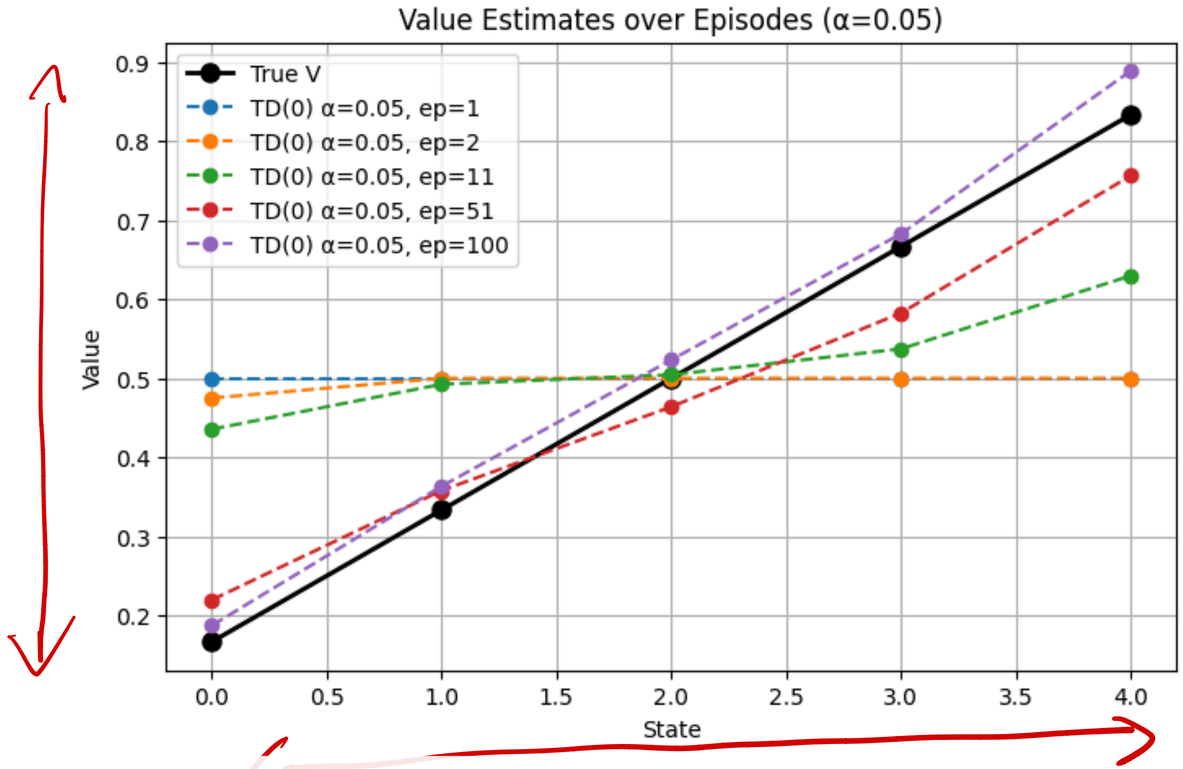
- No actions.
- Start in state C.
- Equal probability of transitioning either left or right at each step.
- Absorbing (terminal) states on the left and the right.
- Reward +1 for reaching the right.
- Reward 0 for everything else.
- No discounting (i.e. $\gamma = 1$).



TD(0) example

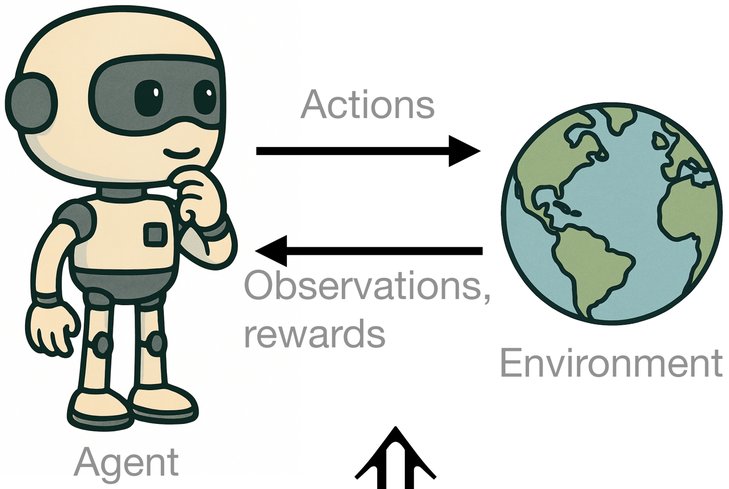


Experiment setup from Sutton and Barto (2018) pg. 125.



- No actions.
- Start in state C.
- Equal probability of transitioning either left or right at each step.
- Absorbing (terminal) states on the left and the right.
- Reward +1 for reaching the right.
- Reward 0 for everything else.
- No discounting (i.e. $\gamma = 1$).

TD(0)-based generalized policy iteration



$$\tau^1 = s_0^1, a_0^1, r_0^1, s_1^1, a_1^1, r_1^1, \ldots, s_{T_1}^1$$

$$\tau^2 = s_0^2, a_0^2, r_0^2, s_1^2, a_1^2, r_1^2, \ldots, s_{T_2}^2$$

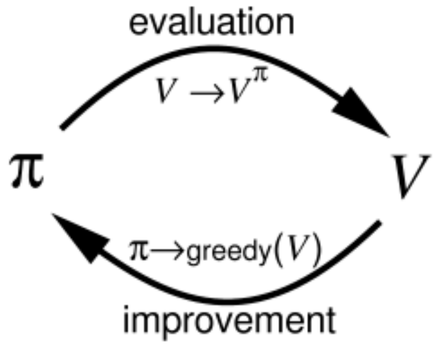
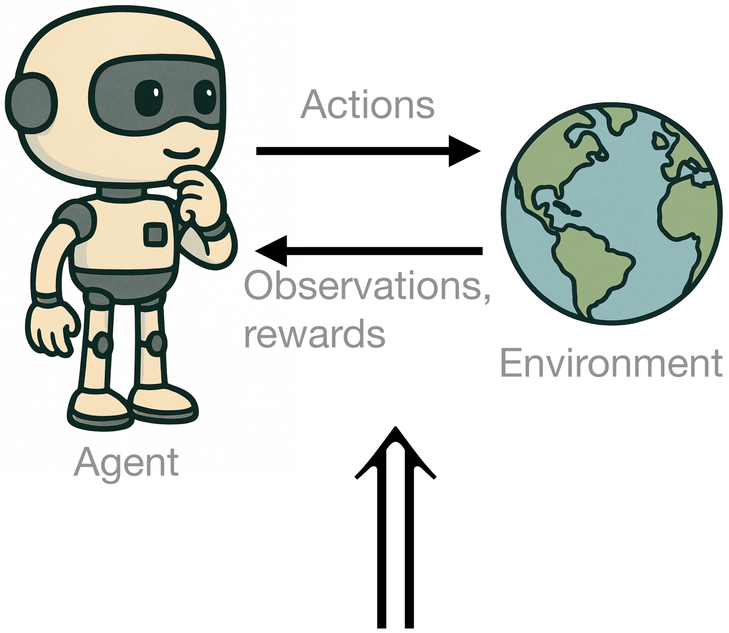


Figure courtesy of Sutton & Barto (2018)

Use estimate of $Q(s, a)$ to improve $\pi \to \pi'$

Monte Carlo estimation of $\hat{Q}(s, a)$

TD(0)-based generalized policy iteration



$$\tau^1 = s_0^1, a_0^1, r_0^1, s_1^1, a_1^1, r_1^1, \ldots, s_{T_1}^1$$

$$\tau^2 = s_0^2, a_0^2, r_0^2, s_1^2, a_1^2, r_1^2, \ldots, s_{T_2}^2$$

One-step transition + reward

$$s_t, a_t, r_t, s_{t+1}, a_{t+1}$$

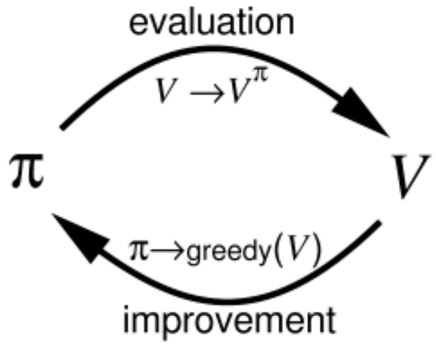


Figure courtesy of Sutton & Barto (2018)

Use estimate of $Q(s, a)$ to improve $\pi \to \pi'$

~~Monte Carlo~~ **TD(0)** estimation of $\hat{Q}(s, a)$

Recall: the need for $\hat{Q}^{\pi}(s, a)$ for extracting greedy policies without MDP models?

Given the MDP model, we could compute a policy improvement step:

$$\pi'(s) = \arg\max_{a \in A} \left[R(s, a) + \gamma \sum_{s' \in S} T(s' \mid s, a) \hat{V}^{\pi}(s) \right]$$

Recall: the need for $\hat{Q}^{\pi}(s, a)$ for extracting greedy policies without MDP models?

Given the MDP model, we could compute a policy improvement step:

$$\pi'(s) = \arg\max_{a \in A} \left[R(s, a) + \gamma \sum_{s' \in S} T(s' \mid s, a) \hat{V}^{\pi}(s) \right]$$

What do we do when we can't evaluate $R(s, a)$ and $T(s' \mid s, a)$?

Recall: the need for $\hat{Q}^{\pi}(s, a)$ for extracting greedy policies without MDP models?

Given the MDP model, we could compute a policy improvement step:

$$\pi'(s) = \arg\max_{a \in A} \left[R(s, a) + \gamma \sum_{s' \in S} T(s' \mid s, a) \hat{V}^{\pi}(s) \right]$$

What do we do when we can't evaluate $R(s, a)$ and $T(s' \mid s, a)$?

We have to estimate state-action values instead: $\hat{Q}(s, a)$

Then, our policy improvement step is as simple as $\pi'(s) = \arg\max_{a \in A} \hat{Q}(s, a)$

SARSA: On-policy TD control

Idea: Do generalized policy iteration, like with MC control, but now use TD(0) value estimation instead.

How can we estimate the state-action value function $\hat{Q}^{\pi}(s, a)$ instead of $\hat{V}^{\pi}(s)$?

SARSA: On-policy TD control

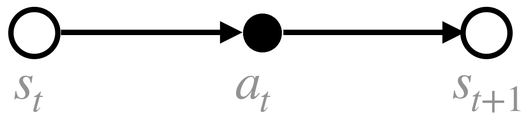
Idea: Do generalized policy iteration, like with MC control, but now use TD(0) value estimation instead.

How can we estimate the state-action value function $\hat{Q}^\pi(s, a)$ instead of $\hat{V}^\pi(s)$?

$\underline{\hat{V}^\pi(s)}$

Execute policy π in the environment. After every step (s_t, a_t, r_t, s_{t+1}), form a target and make an update of $\hat{V}^\pi(s)$.

$$\hat{V}^\pi(s_t) = \hat{V}^\pi(s_t) + \alpha \left(r_t + \gamma \hat{V}^\pi(s_{t+1}) - \hat{V}^\pi(s_t) \right)$$



SARSA: On-policy TD control

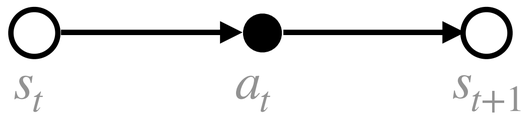
Idea: Do generalized policy iteration, like with MC control, but now use TD(0) value estimation instead.

How can we estimate the state-action value function $\hat{Q}^\pi(s,a)$ instead of $\hat{V}^\pi(s)$?

$\hat{V}^\pi(s)$

Execute policy π in the environment. After every step (s_t, a_t, r_t, s_{t+1}), form a target and make an update of $\hat{V}^\pi(s)$.

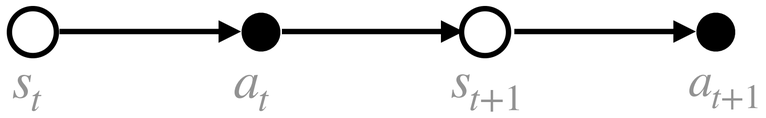
$$\hat{V}^\pi(s_t) = \hat{V}^\pi(s_t) + \alpha\left(r_t + \gamma\hat{V}^\pi(s_{t+1}) - \hat{V}^\pi(s_t)\right)$$



$\hat{Q}^\pi(s,a)$

Execute policy π in the environment. After every step $(s_t, a_t, r_t, s_{t+1}, a_{t+1})$, form a target and make an update of $\hat{Q}^\pi(s,a)$.

$$\hat{Q}(s_t, a_t) = \hat{Q}(s_t, a_t) + \alpha\left(r_t + \gamma\hat{Q}(s_{t+1}, a_{t+1}) - \hat{Q}(s_t, a_t)\right)$$



SARSA: On-policy TD control

Idea: Do generalized policy iteration, like with MC control, but now use TD(0) value estimation instead.

After every step $(s_t, a_t, r_t, s_{t+1}, a_{t+1})$, form a target and make an update of $\hat{Q}(s, a)$.

S A R S A

$$\hat{Q}(s_t, a_t) = \hat{Q}(s_t, a_t) + \alpha \left(r_t + \gamma \hat{Q}(s_{t+1}, a_{t+1}) - \hat{Q}(s_t, a_t) \right)$$



SARSA: On-policy TD control

Idea: Do generalized policy iteration, like with MC control, but now use TD(0) value estimation instead.

After every step $(s_t, a_t, r_t, s_{t+1}, a_{t+1})$, form a target and make an update of $\hat{Q}(s, a)$.

S A R S A

$$\hat{Q}(s_t, a_t) = \hat{Q}(s_t, a_t) + \alpha \left(r_t + \gamma \hat{Q}(s_{t+1}, a_{t+1}) - \hat{Q}(s_t, a_t) \right)$$



GPI: Continuously iterate between estimating $\hat{Q}(s, a)$ from samples, and using the estimate to define ϵ-greedy actions in the environment.

$$\pi(a \mid s) = \begin{cases} \text{Uniform(a)}, & \text{w.p. } \epsilon \\ \arg\max_{a \in A} \hat{Q}(s, a) & \text{w.p. } 1 - \epsilon \end{cases}$$

SARSA: On-policy TD control

Idea: Do generalized policy iteration, like with MC control, but now use TD(0) value estimation instead.

After every step $(s_t, a_t, r_t, s_{t+1}, a_{t+1})$, form a target and make an update of $\hat{Q}(s, a)$.

S A R S A

$$\hat{Q}(s_t, a_t) = \hat{Q}(s_t, a_t) + \alpha \left(r_t + \gamma \hat{Q}(s_{t+1}, a_{t+1}) - \hat{Q}(s_t, a_t) \right)$$



GPI: Continuously iterate between estimating $\hat{Q}(s, a)$ from samples,
and using the estimate to define ϵ-greedy actions in the environment.

$$\pi(a \mid s) = \begin{cases} \text{Uniform(a)}, & \text{w.p. } \epsilon \\ \arg\max_{a \in A} \hat{Q}(s, a) & \text{w.p. } 1 - \epsilon \end{cases}$$

Sarsa (on-policy TD control) for estimating $Q \approx q_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$
Initialize $Q(s, a)$, for all $s \in \mathcal{S}^+, a \in \mathcal{A}(s)$, arbitrarily except that $Q(terminal, \cdot) = 0$

Loop for each episode:
  Initialize S
  Choose A from S using policy derived from Q (e.g., ε-greedy)
  Loop for each step of episode:
    Take action A, observe R, S'
    Choose A' from S' using policy derived from Q (e.g., ε-greedy)
    $Q(S, A) \leftarrow Q(S, A) + \alpha\big[R + \gamma Q(S', A') - Q(S, A)\big]$
    $S \leftarrow S'$; $A \leftarrow A'$;
  until S is terminal

Recall our running gridworld example



Goal state
+1 reward
absorbing

Every other step:
-0.1 reward

Dynamics:
“Slip probability” of 0.3

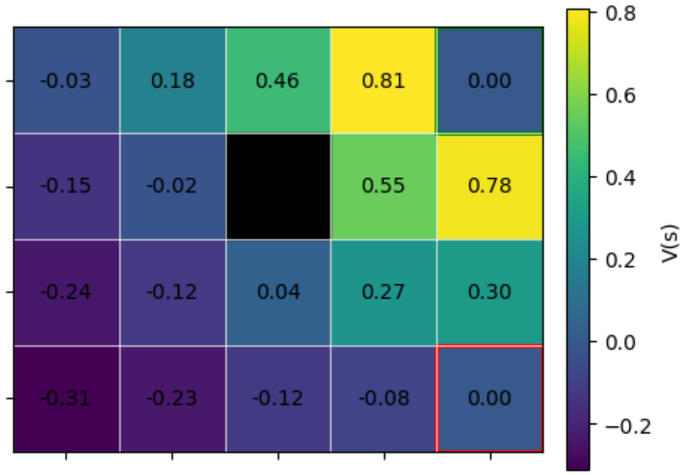
Initial state

Sink state
-1 reward
absorbing

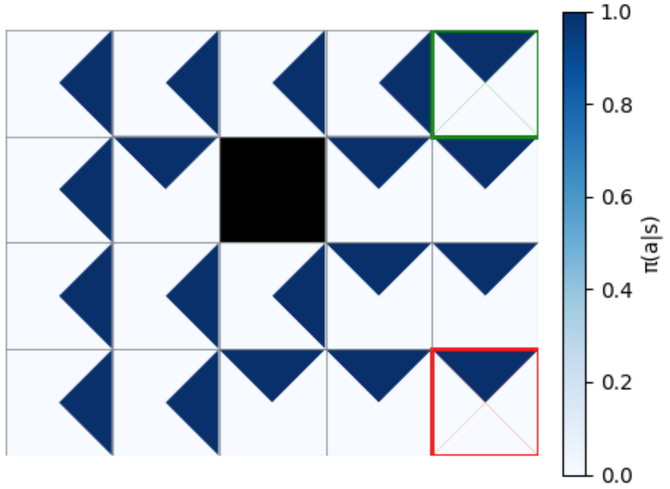
Discount factor $\gamma = 0.9$

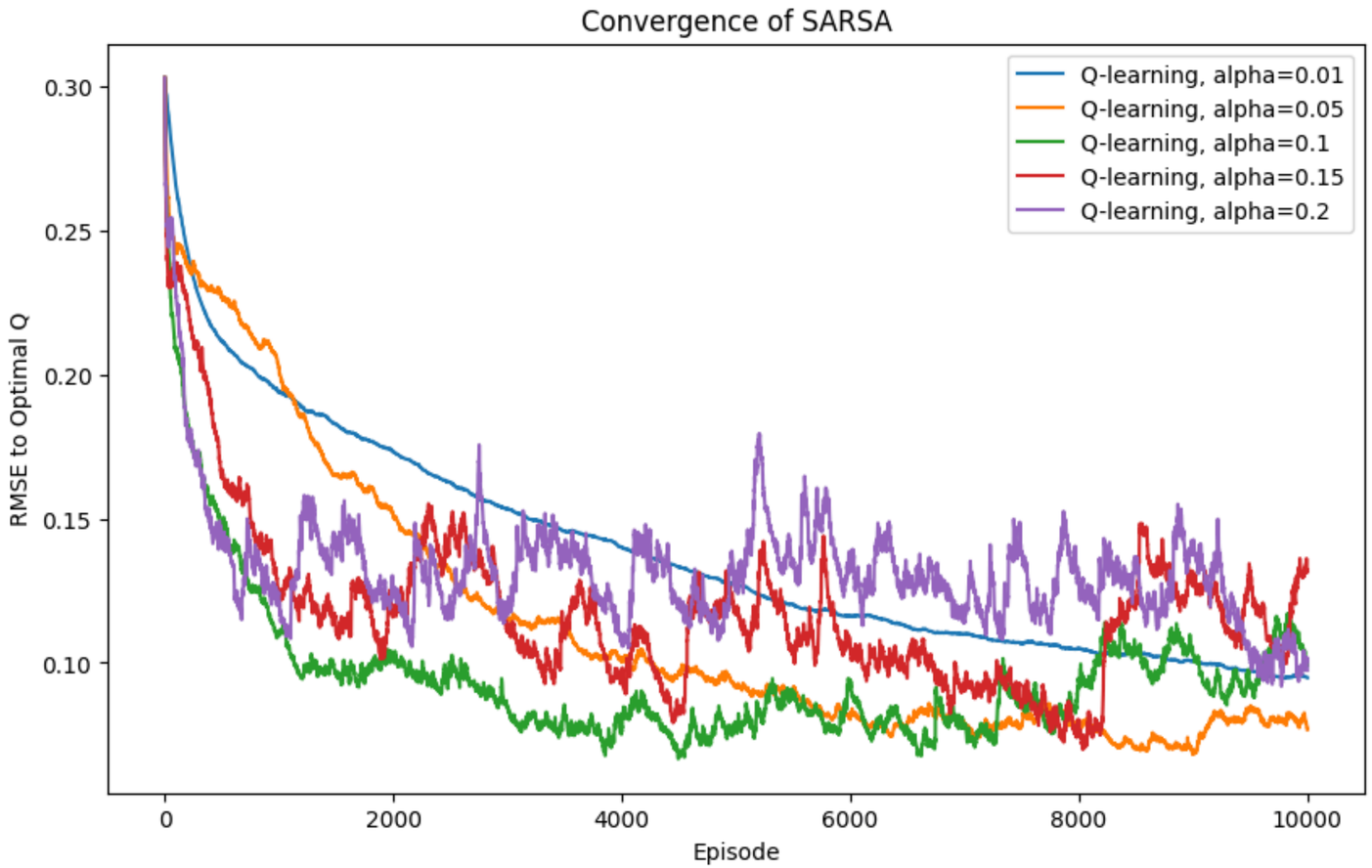
Running gridworld example: SARSA results

Optimal $V^*(s)$, calculated with DP



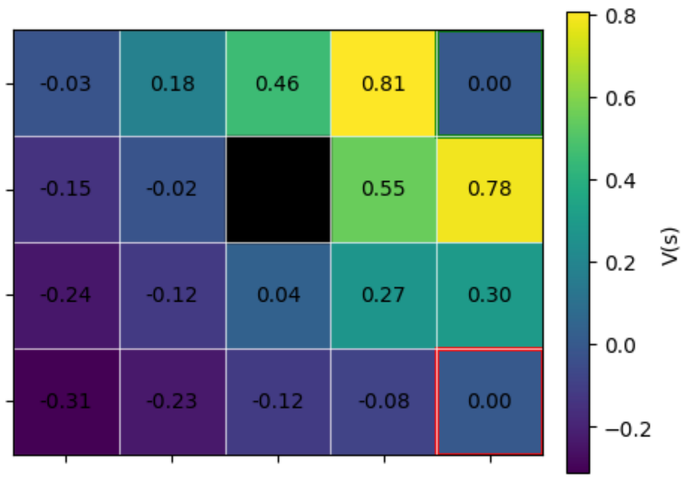
Optimal policy π^*, calculated with DP



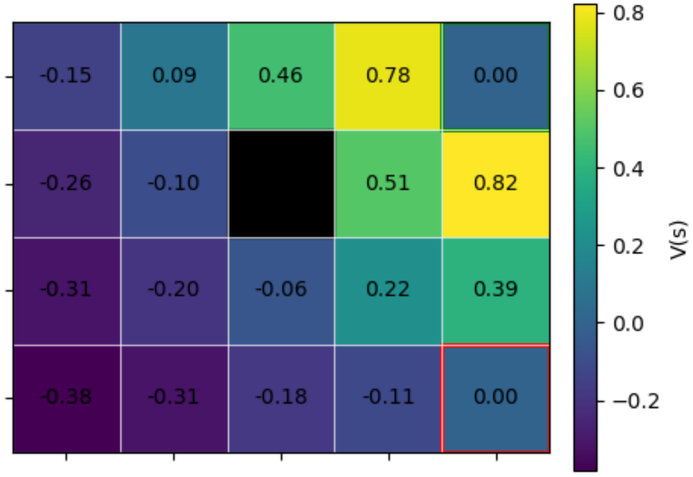


Running gridworld example: SARSA results

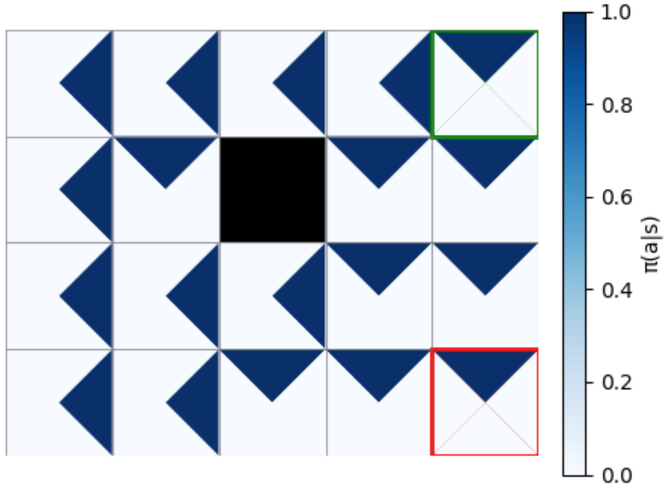
Optimal $V^*(s)$, calculated with DP



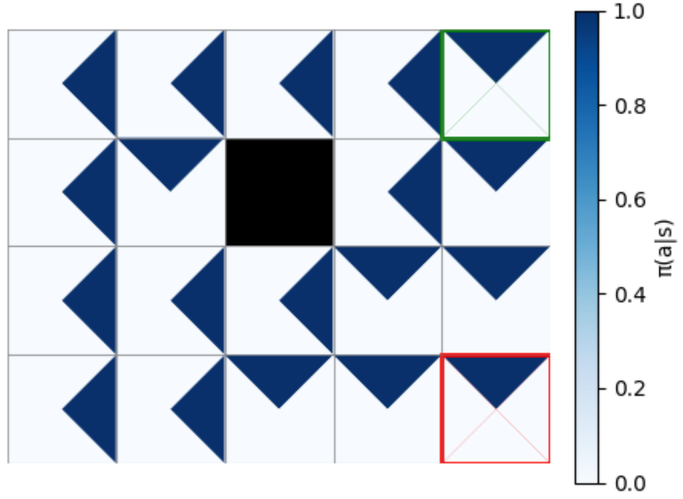
Optimal $V^*(s)$, calculated with SARSA



Optimal policy π^*, calculated with DP



Optimal policy π^*, calculated with SARSA



Q-learning: Off-policy TD control

Note: Key idea, directly approximate the *optimal* state-action value function $\hat{Q}^*(s, a)$,
Instead of approximating the value function for the current policy.

Q-learning: Off-policy TD control

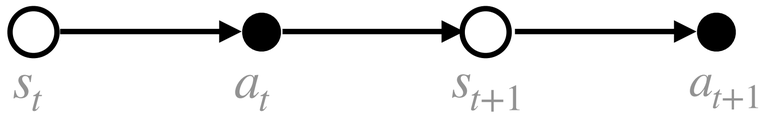
Note: Key idea, directly approximate the *optimal* state-action value function $\hat{Q}^*(s, a)$, Instead of approximating the value function for the current policy.

SARSA:

After every step $(s_t, a_t, r_t, s_{t+1}, a_{t+1})$, form a target and make an update of $\hat{Q}(s, a)$.

S A R S A

$$\hat{Q}(s_t, a_t) = \hat{Q}(s_t, a_t) + \alpha \left(r_t + \gamma \hat{Q}(s_{t+1}, a_{t+1}) - \hat{Q}(s_t, a_t) \right)$$



Q-learning: Off-policy TD control

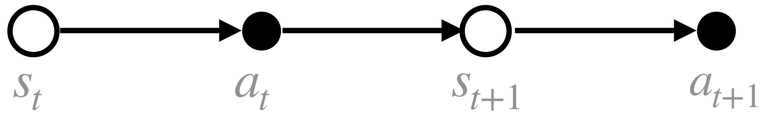
Note: Key idea, directly approximate the *optimal* state-action value function $\hat{Q}^*(s, a)$, Instead of approximating the value function for the current policy.

SARSA:

After every step $(s_t, a_t, r_t, s_{t+1}, a_{t+1})$, form a target and make an update of $\hat{Q}(s, a)$.

S A R S A

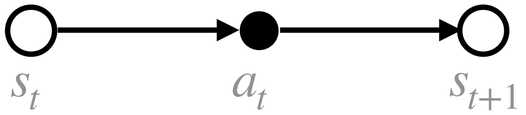
$$\hat{Q}(s_t, a_t) = \hat{Q}(s_t, a_t) + \alpha \left(r_t + \gamma \hat{Q}(s_{t+1}, a_{t+1}) - \hat{Q}(s_t, a_t) \right)$$



Q-learning:

After every step $(s_t, a_t, r_t, s_{t+1}, \cancel{a_{t+1}})$, form a target using the greedy action at time $t+1$, given the current $\hat{Q}(s, a)$.

$$\hat{Q}(s_t, a_t) = \hat{Q}(s_t, a_t) + \alpha \left(r_t + \gamma \max_{a \in A} \hat{Q}(s_{t+1}, a) - \hat{Q}(s_t, a_t) \right)$$



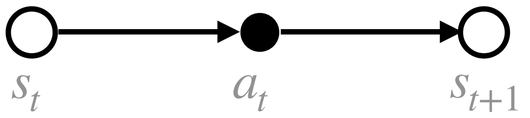
Q-learning: Off-policy TD control

Note: Key idea, directly approximate the *optimal* state-action value function $\hat{Q}^*(s, a)$, Instead of approximating the value function for the current policy.

Q-learning:

After every step $(s_t, a_t, r_t, s_{t+1}, a_{t+1})$, form a target using the greedy action at time $t+1$, given the current $\hat{Q}(s, a)$.

$$\hat{Q}(s_t, a_t) = \hat{Q}(s_t, a_t) + \alpha \left(r_t + \gamma \max_{a \in A} \hat{Q}(s_{t+1}, a) - \hat{Q}(s_t, a_t) \right)$$



Q-learning (off-policy TD control) for estimating $\pi \approx \pi_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$
Initialize $Q(s, a)$, for all $s \in \mathcal{S}^+, a \in \mathcal{A}(s)$, arbitrarily except that $Q(terminal, \cdot) = 0$

Loop for each episode:
  Initialize S
  Loop for each step of episode:
    Choose A from S using policy derived from Q (e.g., ε-greedy)
    Take action A, observe R, S'
    $Q(S, A) \leftarrow Q(S, A) + \alpha\big[R + \gamma \max_a Q(S', a) - Q(S, A)\big]$
    $S \leftarrow S'$
  until S is terminal

Does the Q-learning update remind you of anything?

Q-learning: $$\hat{Q}(s_t, a_t) = \hat{Q}(s_t, a_t) + \alpha \left(r_t + \gamma \max_{a \in A} \hat{Q}(s_{t+1}, a) - \hat{Q}(s_t, a_t) \right)$$

Value iteration! Tabular Q-learning is a stochastic, online version of value iteration.

Bellman optimality: $Q^*(s,a) = R(s,a) + \gamma \sum_{s' \in S} T(s'|s,a) \max_{a'} Q^*(s',a')$

- we have to know T, R, ...
- loop over all states.

expectation.

Q-learning does the same thing, but estimates the expectation using a running estimator from samples in the environment.



How else is Q-learning different from value iteration?

VI is a batch process:

inner loop: iterates over all state-action pairs and applies Bellman update to get one single update over current estimate

outer loop: iteratively applies the inner loop until convergence.

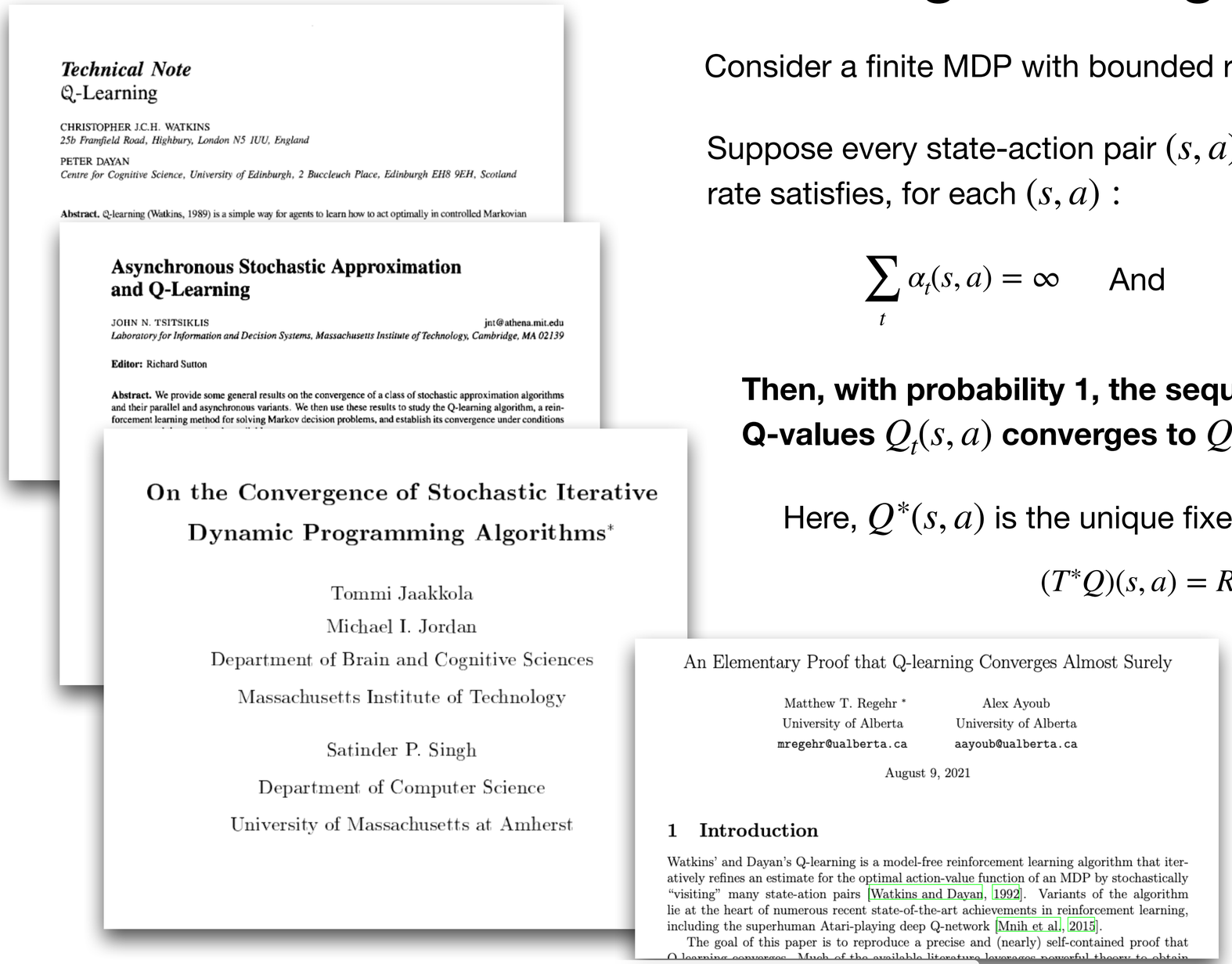
Q-learning doesn't separate the two loops.

↳ applies "inner loop" only locally at sampled transitions.

↳ taking small noisy steps towards the fixed point Q^* that we care about.



Q-Learning convergence



Technical Note
Q-Learning

CHRISTOPHER J.C.H. WATKINS
25b Framfield Road, Highbury, London N5 1UU, England

PETER DAYAN
Centre for Cognitive Science, University of Edinburgh, 2 Buccleuch Place, Edinburgh EH8 9EH, Scotland

Abstract. Q-learning (Watkins, 1989) is a simple way for agents to learn how to act optimally in controlled Markovian

Asynchronous Stochastic Approximation and Q-Learning

JOHN N. TSITSIKLIS jnt@athena.mit.edu
Laboratory for Information and Decision Systems, Massachusetts Institute of Technology, Cambridge, MA 02139

Editor: Richard Sutton

Abstract. We provide some general results on the convergence of a class of stochastic approximation algorithms and their parallel and asynchronous variants. We then use these results to study the Q-learning algorithm, a reinforcement learning method for solving Markov decision problems, and establish its convergence under conditions

On the Convergence of Stochastic Iterative Dynamic Programming Algorithms*

Tommi Jaakkola
Michael I. Jordan
Department of Brain and Cognitive Sciences
Massachusetts Institute of Technology

Satinder P. Singh
Department of Computer Science
University of Massachusetts at Amherst

An Elementary Proof that Q-learning Converges Almost Surely

Matthew T. Regehr * Alex Ayoub
University of Alberta University of Alberta
mregehr@ualberta.ca aayoub@ualberta.ca

August 9, 2021

1 Introduction

Watkins' and Dayan's Q-learning is a model-free reinforcement learning algorithm that iteratively refines an estimate for the optimal action-value function of an MDP by stochastically "visiting" many state-ation pairs [Watkins and Dayan, 1992]. Variants of the algorithm lie at the heart of numerous recent state-of-the-art achievements in reinforcement learning, including the superhuman Atari-playing deep Q-network [Mnih et al., 2015].

The goal of this paper is to reproduce a precise and (nearly) self-contained proof that

Consider a finite MDP with bounded rewards $R(s,a)$ and a discount factor $0 \leq \gamma < 1$.

Suppose every state-action pair (s,a) is visited infinitely often, and the learning rate satisfies, for each (s,a) :

$$\sum_t \alpha_t(s,a) = \infty \quad \text{And} \quad \sum_t \alpha_t(s,a)^2 < \infty$$ Robbins-Monro conditions

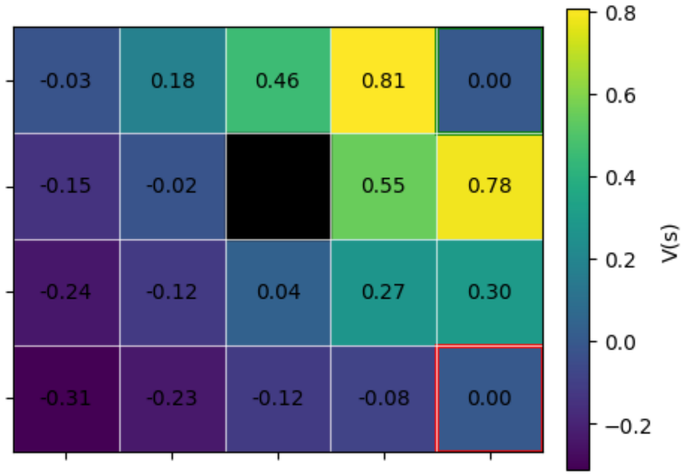
Then, with probability 1, the sequence of approximated Q-values $Q_t(s,a)$ converges to $Q^*(s,a)$.

Here, $Q^*(s,a)$ is the unique fixed point of the Bellman optimality operator:

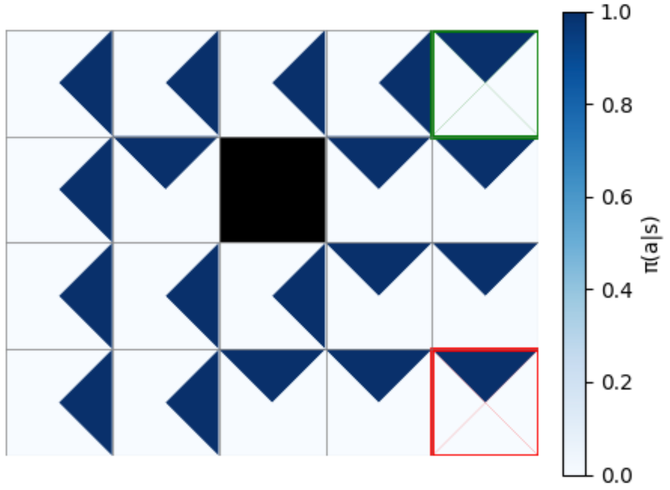
$$(T^*Q)(s,a) = R(s,a) + \gamma \max_{a' \in A} \mathbb{E}_{S' \sim T(\cdot|s,a)} \left[Q(S', a') \right]$$

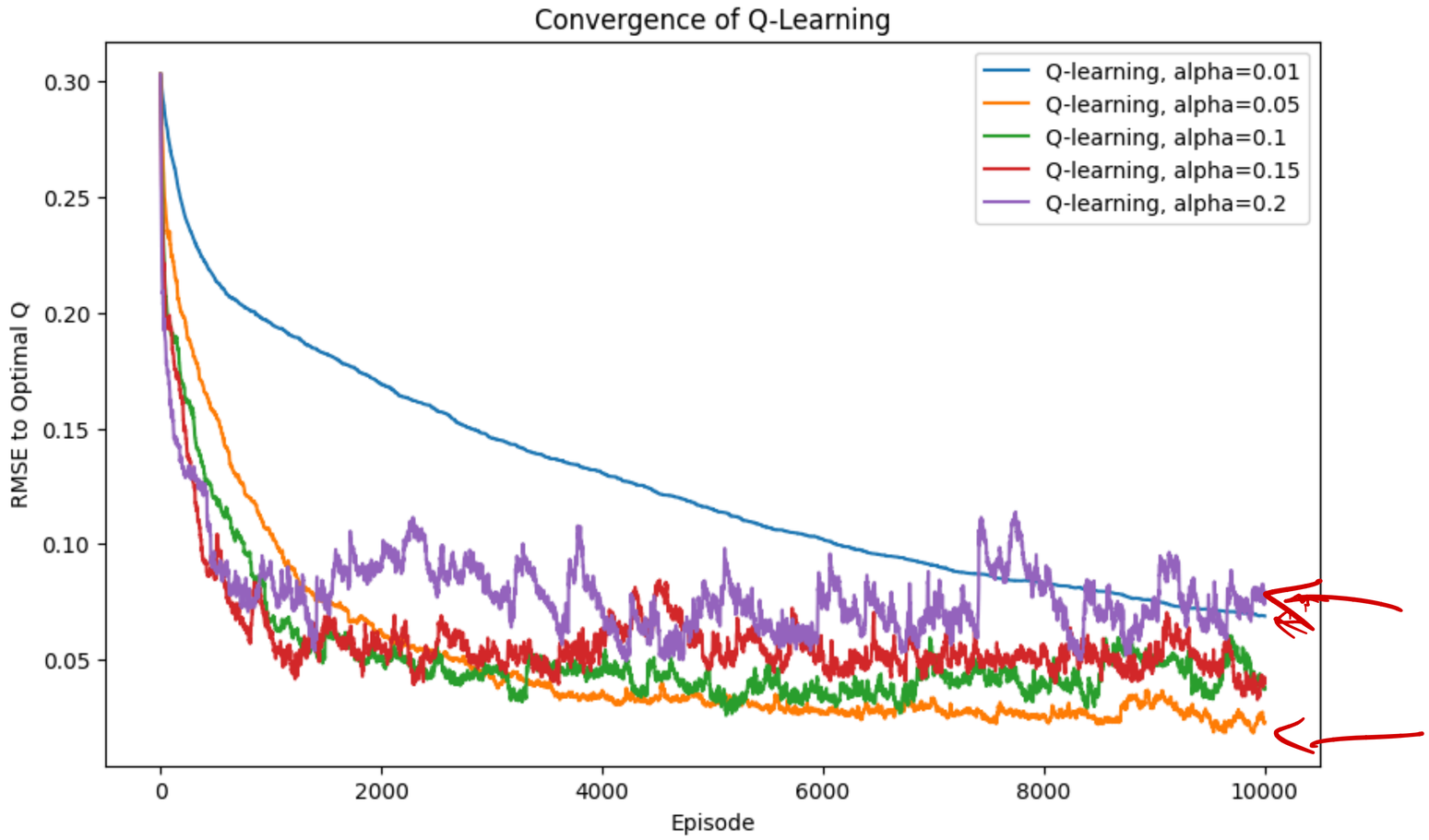
Running gridworld example: Q-Learning results

Optimal $V^*(s)$, calculated with DP



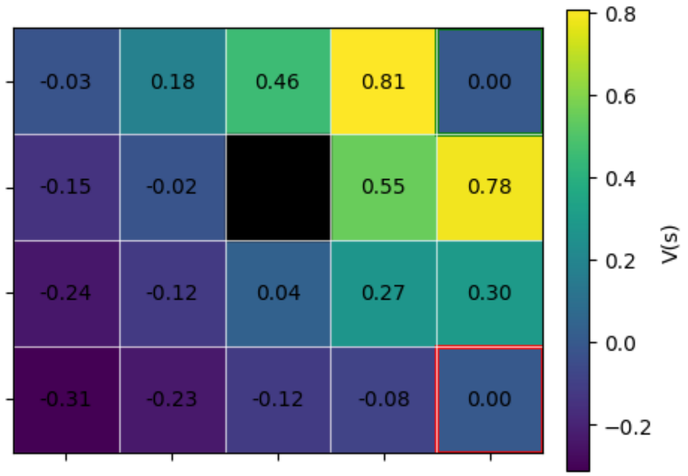
Optimal policy π^*, calculated with DP



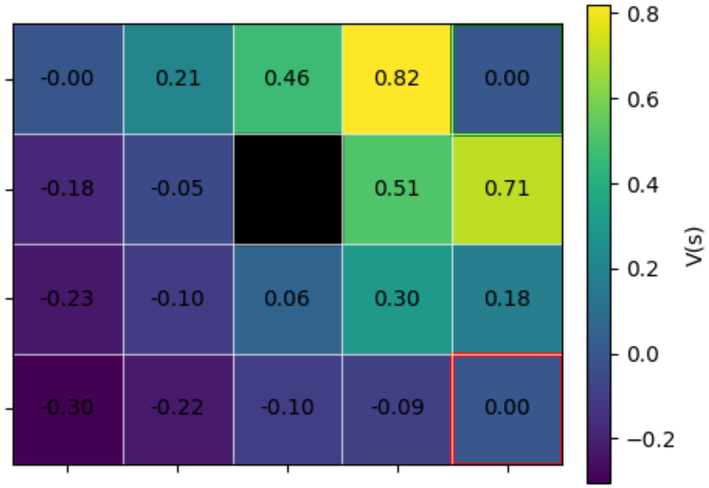


Running gridworld example: Q-Learning results

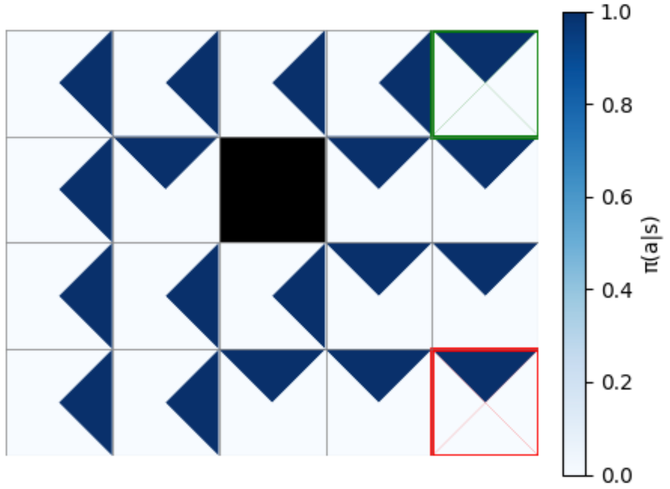
Optimal $V^*(s)$, calculated with DP



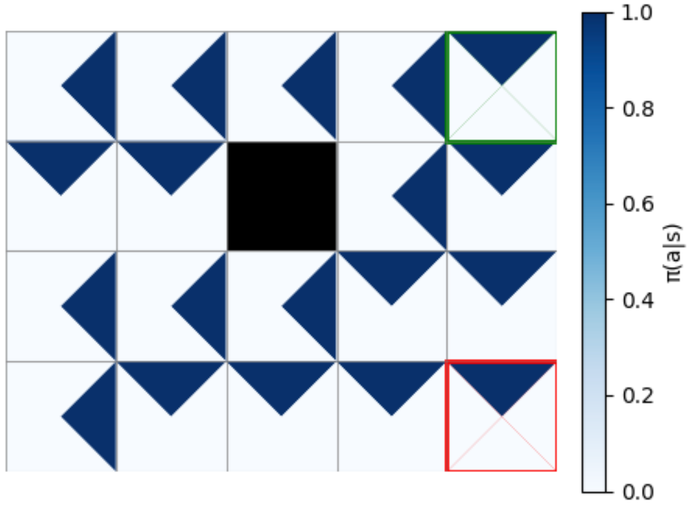
Optimal $V^*(s)$, calculated with Q-Learning



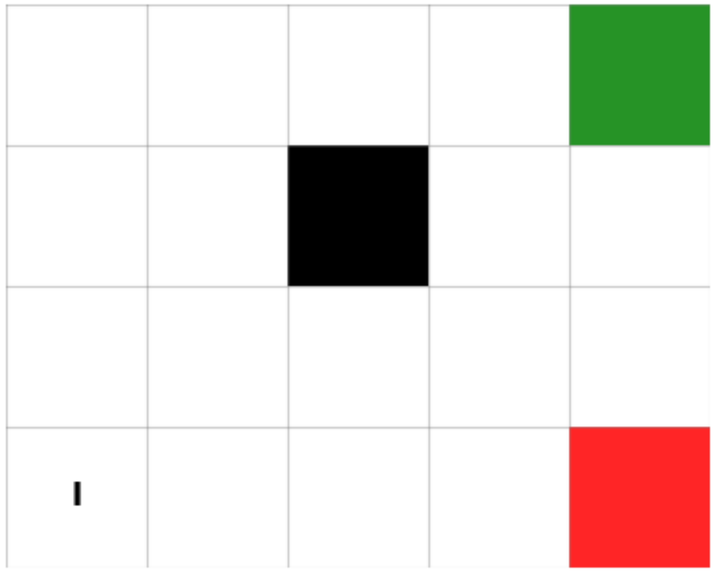
Optimal policy π^*, calculated with DP

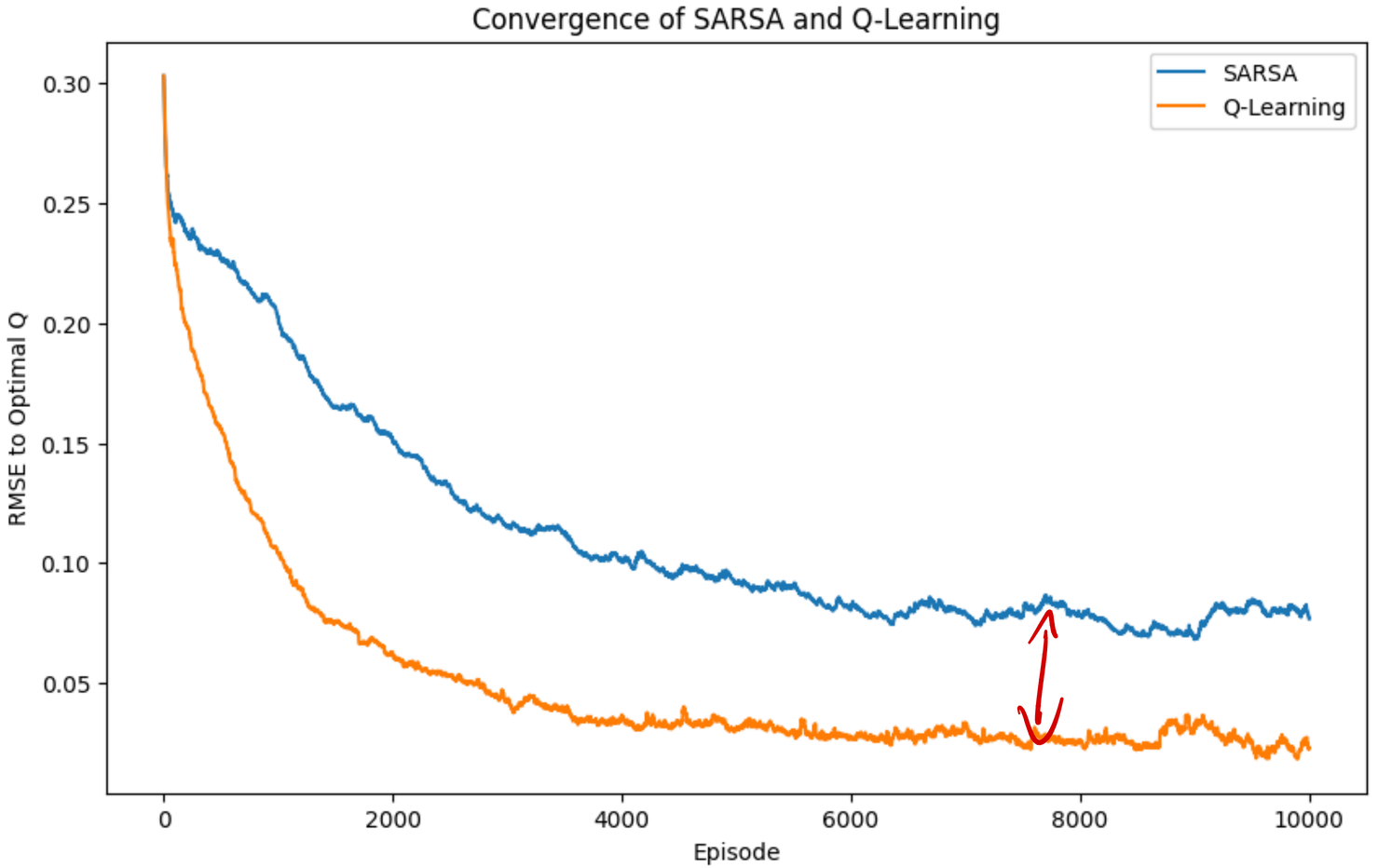


Optimal policy π^*, calculated with Q-Learning



Comparing Q-Learning performance with SARSA





Why does Q-Learning empirically outperform SARSA?

Speculation:

- Q-learning being off-policy means backups use max operator, we are moving straight towards optimal values and faster.
- Q-learning target directly estimates Q^*, whereas SARSA estimates ϵ-greedy policy w.r.t. current $\hat{Q}$ estimate.



Why is SARSA considered “on-policy” while Q-learning is “off-policy”?

SARSA $\hat{Q}(s,a) \leftarrow \hat{Q}(s,a) + \alpha(r + \gamma \hat{Q}(s',a') - \hat{Q}(s,a))$

a' is sampled from current policy π

Q-learning $\hat{Q}(s,a) \leftarrow \hat{Q}(s,a) + \alpha(r + \gamma \max_{a'} \hat{Q}(s',a') - \hat{Q}(s,a))$

maximum over actions in current estimate of Q^*.



Intuitively, what's the difference between SARSA and Q-learning?

SARSA
↳ what will happen, given the current policy.

Q-learning
↳ Learns what's best under ideal future actions.

Is Q-learning strictly "better" than SARSA?

↳ No!
- Maximization bias
  - Q-learning optimistically learns values for ideal future actions.
    But during exploration, that's not what our behavioral policy is doing.



The cliff-walking example: Q-Learning vs. SARSA

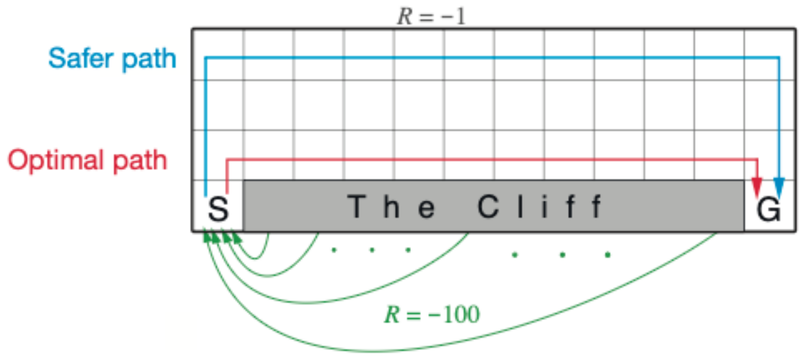


Figure courtesy of Sutton and Barto (2018) pg. 132.

Reward is -1 on all transitions, except in states marked “cliff”, where rewards are -100.

Deterministic dynamics.

Optimal policy: The shortest path, walking along the edge of the cliff.

Safer, sub-optimal policy: The longer path, walking away from the cliff edge.

The cliff-walking example: Q-Learning vs. SARSA

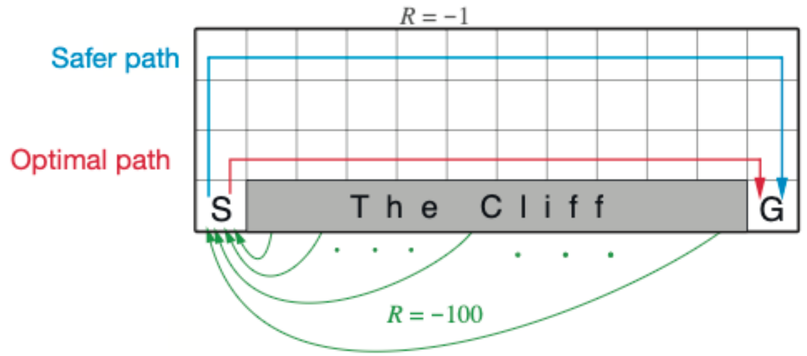


Figure courtesy of Sutton and Barto (2018) pg. 132.

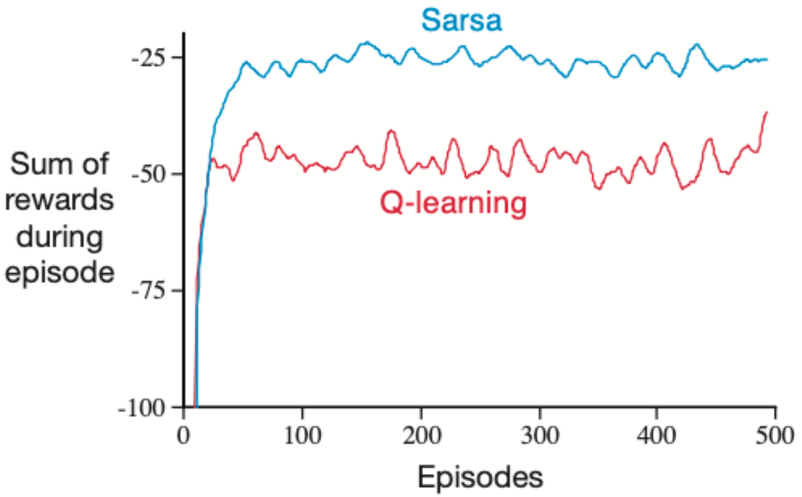


Figure courtesy of Sutton and Barto (2018) pg. 132.

Reward is -1 on all transitions, except in states marked “cliff”, where rewards are -100.

Deterministic dynamics.

Optimal policy: The shortest path, walking along the edge of the cliff.

Safer, sub-optimal policy: The longer path, walking away from the cliff edge.

The cliff-walking example: Q-Learning vs. SARSA

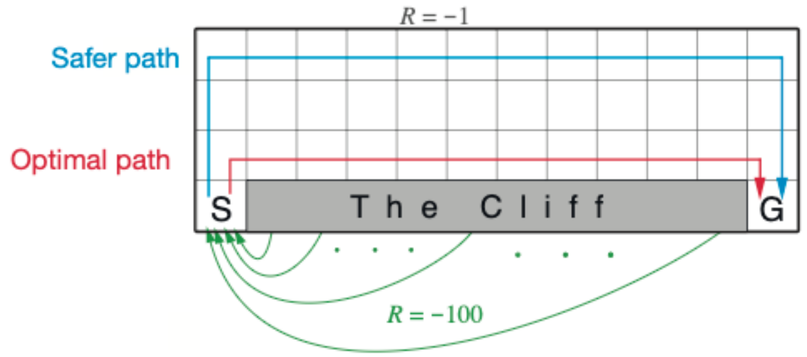


Figure courtesy of Sutton and Barto (2018) pg. 132.

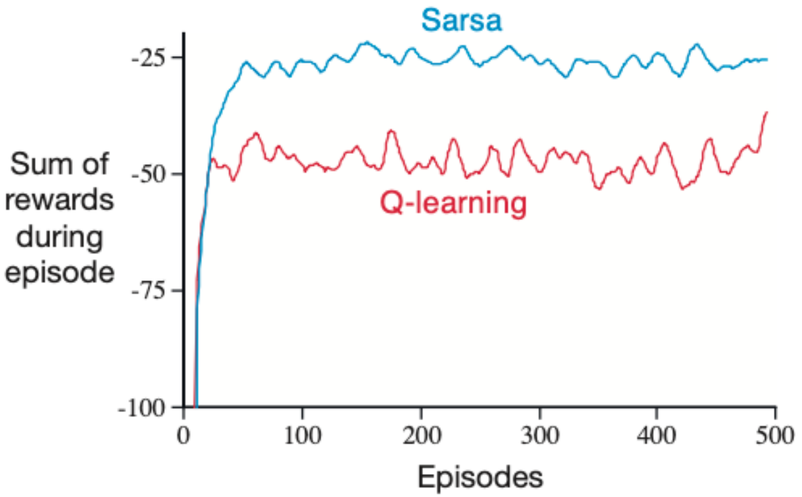


Figure courtesy of Sutton and Barto (2018) pg. 132.

Reward is -1 on all transitions, except in states marked “cliff”, where rewards are -100.

Deterministic dynamics.

Optimal policy: The shortest path, walking along the edge of the cliff.

Safer, sub-optimal policy: The longer path, walking away from the cliff edge.

Q-learning learns the **optimal policy**, by assuming ideal future actions.

However, when operating with ϵ-greedy actions, this results in the agent frequently stepping on the cliff.

The cliff-walking example: Q-Learning vs. SARSA

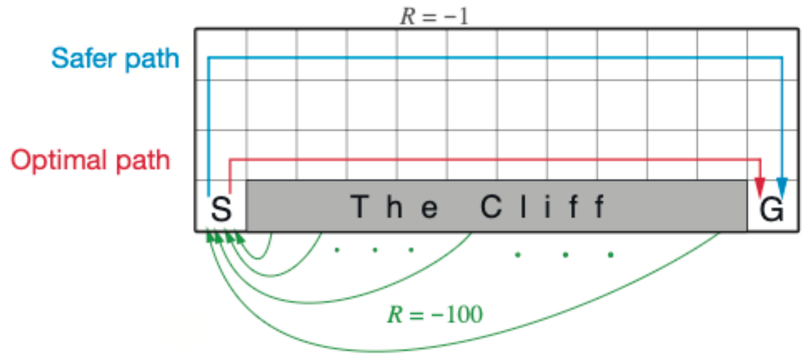


Figure courtesy of Sutton and Barto (2018) pg. 132.

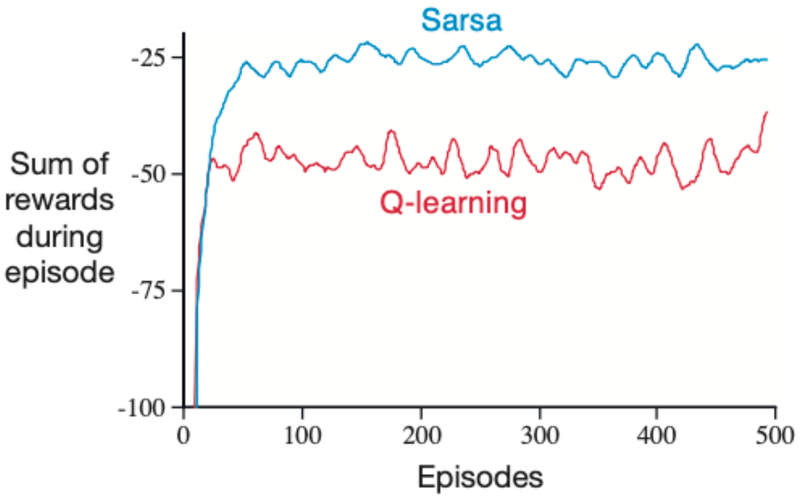


Figure courtesy of Sutton and Barto (2018) pg. 132.

Reward is -1 on all transitions, except in states marked “cliff”, where rewards are -100.

Deterministic dynamics.

Optimal policy: The shortest path, walking along the edge of the cliff.

Safer, sub-optimal policy: The longer path, walking away from the cliff edge.

Q-learning learns the **optimal policy**, by assuming ideal future actions.

However, when operating with ϵ-greedy actions, this results in the agent frequently stepping on the cliff.

SARSA learns the **safe policy**, by using on-policy ϵ -greedy action selection to define its learning target.

As a result, it achieves higher rewards when learning in the environment (in this example).

Maximization bias

step → s_t, a_t, r_t, s_{t+1}

Q-Learning update: $$\hat{Q}(s_t, a_t) \leftarrow \hat{Q}(s_t, a_t) + \alpha \left(r_t + \gamma \max_{a \in A} \hat{Q}(s_{t+1}, a) - \hat{Q}(s_t, a_t) \right)$$

Update rule moves current estimate $\hat{Q}(s, a)$ towards the target $r_t + \gamma \max_{a \in A} \hat{Q}(s, a)$.

Note: Our target is defined using the current estimate $\hat{Q}(s, a)$, instead of the true optimal value function $Q^*(s, a)$.

Maximization bias

Q-Learning update: $$\hat{Q}(s_t, a_t) \leftarrow \hat{Q}(s_t, a_t) + \alpha \left(r_t + \gamma \max_{a \in A} \hat{Q}(s_{t+1}, a) - \hat{Q}(s_t, a_t) \right)$$

Update rule moves current estimate $\hat{Q}(s, a)$ towards the target $r_t + \gamma \max_{a \in A} \hat{Q}(s, a)$.

Note: Our target is defined using the current estimate $\hat{Q}(s, a)$, instead of the true optimal value function $Q^*(s, a)$.

This introduces **maximization bias**: We are taking a maximization over *noisy* estimates of the value.
This can bias the results to be higher than they should be, especially early on in training!

Maximization bias

Q-Learning update: $$\hat{Q}(s_t, a_t) \leftarrow \hat{Q}(s_t, a_t) + \alpha \left(r_t + \gamma \max_{a \in A} \hat{Q}(s_{t+1}, a) - \hat{Q}(s_t, a_t) \right)$$

Update rule moves current estimate $\hat{Q}(s, a)$ towards the target $r_t + \gamma \max_{a \in A} \hat{Q}(s, a)$.

Note: Our target is defined using the current estimate $\hat{Q}(s, a)$, instead of the true optimal value function $Q^*(s, a)$.

This introduces **maximization bias**: We are taking a maximization over *noisy* estimates of the value.
This can bias the results to be higher than they should be, especially early on in training!

Intuition: If you take several noisy estimates of numbers, the **maximum** of those estimates is, on average, a little too high. This is because you're likely to pick the one that got lucky noise in the positive direction.

Maximization bias example

Imagine a state s with two actions a_1, a_2.

Assume your imperfect $\hat{Q}(s,a)$ estimates are given by:

$$\hat{Q}(s,a_1) = Q^*(s,a_1) + \epsilon_1 \qquad \hat{Q}(s,a_2) = Q^*(s,a_1) + \epsilon_2$$

where ϵ_1, ϵ_2 are zero-mean Gaussian R.V.s

Maximization bias example

Imagine a state s with two actions a_1, a_2.

Assume your imperfect $\hat{Q}(s,a)$ estimates are given by:

$$\hat{Q}(s,a_1) = Q^*(s,a_1) + \epsilon_1 \qquad \hat{Q}(s,a_2) = Q^*(s,a_2) + \epsilon_2$$

where ϵ_1, ϵ_2 are zero-mean Gaussian R.V.s

The maximization in the Q-learning backup becomes $\max_{a \in \{a_1, a_2\}} \hat{Q}(s,a) = Q^*(s,a) + \max\{\epsilon_1, \epsilon_2\}$

Maximization bias example

Imagine a state s with two actions a_1, a_2.

Assume your imperfect $\hat{Q}(s,a)$ estimates are given by:

$$\hat{Q}(s,a_1) = Q^*(s,a_1) + \epsilon_1 \qquad \hat{Q}(s,a_2) = Q^*(s,a_1) + \epsilon_2$$

where ϵ_1, ϵ_2 are zero-mean Gaussian R.V.s

The maximization in the Q-learning backup becomes $\max_{a \in \{a_1, a_2\}} \hat{Q}(s,a) = Q^*(s,a) + \max\{\epsilon_1, \epsilon_2\}$

But, $\mathbb{E}\left[\max\{\epsilon_1, \epsilon_2\}\right] > \max\{\mathbb{E}[\epsilon_1], \mathbb{E}[\epsilon_1]\} = 0.$ E.g., if $\epsilon_i \sim \mathcal{N}(0,1)$, $\mathbb{E}[max\{\epsilon_1, \epsilon_2\}] \approx 0.56$.

Maximization bias example

Imagine a state s with two actions a_1, a_2.

Assume your imperfect $\hat{Q}(s,a)$ estimates are given by:

$$\hat{Q}(s,a_1) = Q^*(s,a_1) + \epsilon_1 \qquad \hat{Q}(s,a_2) = Q^*(s,a_1) + \epsilon_2$$

where ϵ_1, ϵ_2 are zero-mean Gaussian R.V.s

The maximization in the Q-learning backup becomes $\max_{a \in \{a_1, a_2\}} \hat{Q}(s,a) = Q^*(s,a) + \max\{\epsilon_1, \epsilon_2\}$

But, $\mathbb{E}\left[\max\{\epsilon_1, \epsilon_2\}\right] > \max\{\mathbb{E}[\epsilon_1], \mathbb{E}[\epsilon_1]\} = 0.$ E.g., if $\epsilon_i \sim \mathcal{N}(0,1)$, $\mathbb{E}[max\{\epsilon_1, \epsilon_2\}] \approx 0.56.$

So, the learning target becomes *systematically* overestimated:

$$\mathbb{E}\left[\max_{a \in A} \hat{Q}(s,a)\right] > \max_{a \in A} \mathbb{E}\left[\hat{Q}(s,a)\right].$$

Another maximization bias example

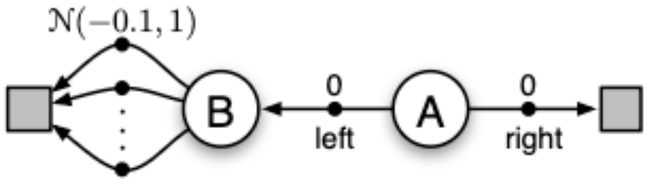


Figure courtesy of Sutton and Barto (2018) pg. 135.

- Initial state A.
- Two actions from A, left or right.
  - Right leads to terminal state with no reward.
  - Left leads to state B.
- From state B, there are MANY available actions, with rewards distributed according to $R \sim \mathcal{N}(-0.1, 1)$.
- So, the expected value of taking action *left* and reaching state B is -0.1, which is lower than the expected value of taking *right* and terminating with reward of 0.0.

Another maximization bias example

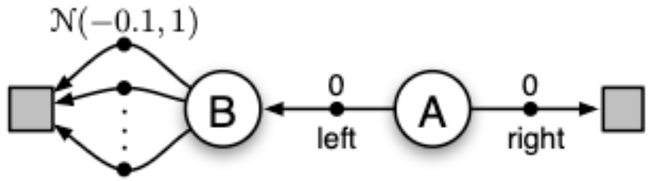
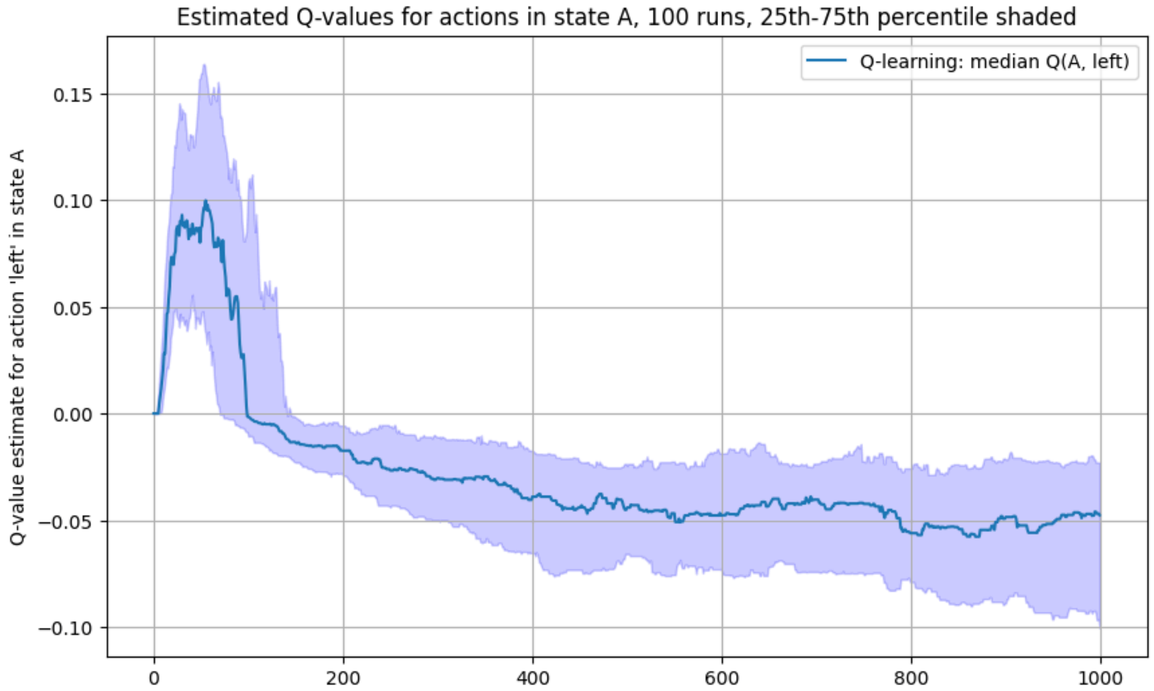
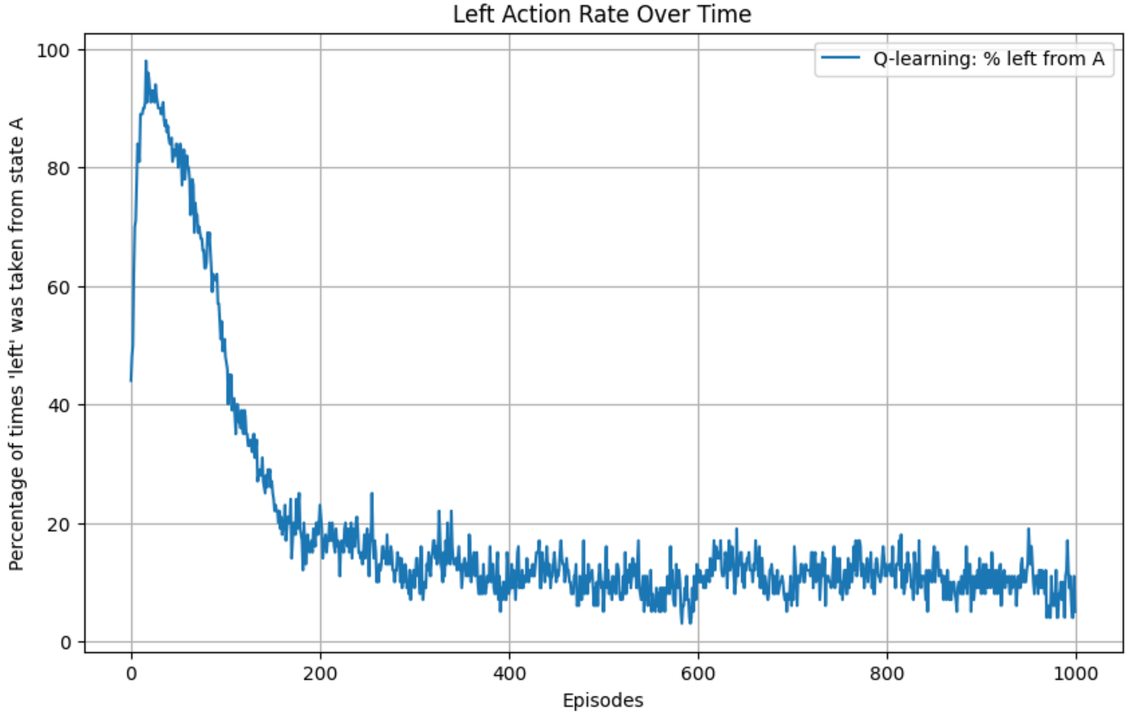


Figure courtesy of Sutton and Barto (2018) pg. 135.

- Initial state A.
- Two actions from A, left or right.
  - Right leads to terminal state with no reward.
  - Left leads to state B.
- From state B, there are MANY available actions, with rewards distributed according to $R \sim \mathcal{N}(-0.1, 1)$.
- So, the expected value of taking action *left* and reaching state B is -0.1, which is lower than the expected value of taking *right* and terminating with reward of 0.0.





Countering maximization bias: double Q-Learning

Intuition: maximization bias comes from using the same $\hat{Q}(s,a)$ to **select the best action**, and to **evaluate its Q-value**. If the *same noise* appears in both roles, it reinforces itself.

Select the best action

$$\hat{Q}(s,a) \leftarrow \hat{Q}(s,a) + \alpha \left[R(s,a) + \gamma \hat{Q}(s', \text{argmax}_{a'} \hat{Q}(s',a')) - \hat{Q}(s,a) \right]$$

Estimate the value of this selection

Countering maximization bias: double Q-Learning

Intuition: maximization bias comes from using the same $\hat{Q}(s,a)$ to **select the best action**, and to **evaluate its Q-value**. If the *same noise* appears in both roles, it reinforces itself.

Select the best action

$$\hat{Q}(s,a) \leftarrow \hat{Q}(s,a) + \alpha \left[R(s,a) + \gamma \hat{Q}(s', \text{argmax}_{a'} \hat{Q}(s',a')) - \hat{Q}(s,a) \right]$$

Estimate the value of this selection

Solution: maintain two estimates $\hat{Q}_A(s,a)$ and $\hat{Q}_B(s,a)$. Each update, randomly select one of the estimates to use for action selection, and the other for action evaluation.

Countering maximization bias: double Q-Learning

Intuition: maximization bias comes from using the same $\hat{Q}(s,a)$ to **select the best action**, and to **evaluate its Q-value**. If the *same noise* appears in both roles, it reinforces itself.

Select the best action

$$\hat{Q}(s,a) \leftarrow \hat{Q}(s,a) + \alpha \left[R(s,a) + \gamma \hat{Q}(s', \text{argmax}_{a'} \hat{Q}(s',a')) - \hat{Q}(s,a) \right]$$

Estimate the value of this selection

Solution: maintain two estimates $\hat{Q}_A(s,a)$ and $\hat{Q}_B(s,a)$. Each update, randomly select one of the estimates to use for action selection, and the other for action evaluation.

That is, each update, choose A or B with 50/50 chance.

If updating A:

$$\hat{Q}_A(s,a) \leftarrow \hat{Q}_A(s,a) + \alpha \left[R(s,a) + \gamma \hat{Q}_B(s', \text{argmax}_{a'} \hat{Q}_A(s',a')) - \hat{Q}_A(s,a) \right]$$

If updating B:

$$\hat{Q}_B(s,a) \leftarrow \hat{Q}_B(s,a) + \alpha \left[R(s,a) + \gamma \hat{Q}_A(s', \text{argmax}_{a'} \hat{Q}_B(s',a')) - \hat{Q}_B(s,a) \right]$$

Countering maximization bias: double Q-Learning

Why should this work??

Intuition: Because the two estimates accumulate independent noise, they don't share the same over-optimistic bumps.

For example, when using $\hat{Q}_A$ to pick an action and $\hat{Q}_B$ to evaluate that action:

- The positive noise in $\hat{Q}_A$ that causes it to pick a particular action doesn't also inflate that action's value according to $\hat{Q}_B$.
- In expectation, we hope that the positive and negative noise will cancel out, and so our Q-Learning target should become unbiased.

Countering maximization bias: double Q-Learning

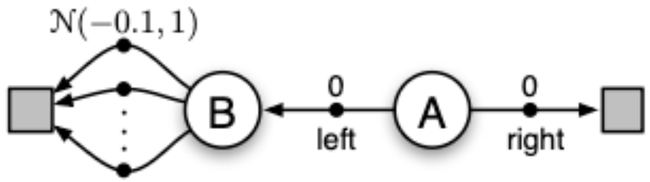


Figure courtesy of Sutton and Barto (2018) pg. 135.

Double Q-learning, for estimating $Q_1 \approx Q_2 \approx q_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$
Initialize $Q_1(s, a)$ and $Q_2(s, a)$, for all $s \in \mathcal{S}^+, a \in \mathcal{A}(s)$, such that $Q(\textit{terminal}, \cdot) = 0$

Loop for each episode:
  Initialize S
  Loop for each step of episode:
    Choose A from S using the policy ε-greedy in $Q_1 + Q_2$
    Take action A, observe R, S'
    With 0.5 probabilility:

$$Q_1(S, A) \leftarrow Q_1(S, A) + \alpha\Big(R + \gamma Q_2\big(S', \arg\max_a Q_1(S', a)\big) - Q_1(S, A)\Big)$$

    else:

$$Q_2(S, A) \leftarrow Q_2(S, A) + \alpha\Big(R + \gamma Q_1\big(S', \arg\max_a Q_2(S', a)\big) - Q_2(S, A)\Big)$$

    $S \leftarrow S'$
  until S is terminal

Countering maximization bias: double Q-Learning

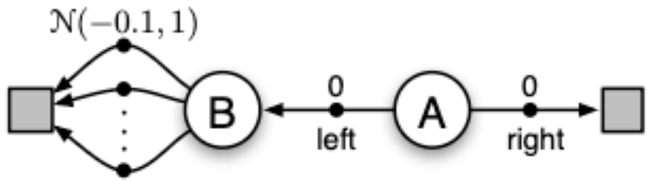


Figure courtesy of Sutton and Barto (2018) pg. 135.

Double Q-learning, for estimating $Q_1 \approx Q_2 \approx q_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$
Initialize $Q_1(s, a)$ and $Q_2(s, a)$, for all $s \in \mathcal{S}^+, a \in \mathcal{A}(s)$, such that $Q(terminal, \cdot) = 0$

Loop for each episode:
  Initialize S
  Loop for each step of episode:
    Choose A from S using the policy ε-greedy in $Q_1 + Q_2$
    Take action A, observe R, S'
    With 0.5 probabilility:

$$Q_1(S, A) \leftarrow Q_1(S, A) + \alpha\Big(R + \gamma Q_2\big(S', \arg\max_a Q_1(S', a)\big) - Q_1(S, A)\Big)$$

    else:

$$Q_2(S, A) \leftarrow Q_2(S, A) + \alpha\Big(R + \gamma Q_1\big(S', \arg\max_a Q_2(S', a)\big) - Q_2(S, A)\Big)$$

    $S \leftarrow S'$
  until S is terminal

